

GuzmanGes

<https://github.com/Iago-3004/guzmanges-proyecto>

Nome Alumno/a: Iago Malvido Guzmán

Curso: 2º DAM

Módulo: *Proyecto Final Ciclo*

Contido

Contido	2
1. Introducción	3
2. Obxectivos	3
3. Situación previa	3
4. Tecnoloxías empregadas	4
5. Solución proposta	4
6. Planificación do proxecto	5
6.1 Presuposto completo (Hardware, software e man de obra)	5
6.2 Diagrama de Gantt	6
6.3 Modelo de desenvolvemento de software a implementar	7
6.4 Análise do proxecto	7
6.4.1 Definición do alcance do proxecto	7
6.4.2 Universo de discurso da base de datos	11
6.4.3 Diagramas Base de Datos	17
6.4.4 Diagrama de casos de uso	19
6.4.5 Descrición dos casos de uso	20
6.4.6 Diagramas de secuencia dos casos de uso	41
6.4.7 Diagrama de clases	49
7. Desenvolvemento e execución	50
7.1. Arquitectura e diagrama de despregamento	50
7.2. Descrición xeral do funcionamento	51
7.3. Detalles técnicos resoltos durante o desenvolvemento	51
7.3.1. Identidade dual para soporte offline	51
7.3.2. Reconciliación de borrados na sincronización incremental	52
7.3.3. Validación oficial de CIF/NIF antes do envío	52
7.3.4. Bugs específicos de iOS	52
7.3.5. Dockerización e despregamento reproducible	53
7.3.6. Documentación da API con OpenAPI/Swagger	53
7.3.7. Integración continua e xeración automática de instaladores	53
8. Conclusións e reflexións	53
8.1. Valoración global do proxecto	53
8.2. Aprendizaxes	54
8.3. Dificultades atopadas	54
8.4. Limitacións actuais	55
8.5. Liñas futuras	55
9. Bibliografía e Webgrafía	55
9.1. Frontend (Flutter, Dart)	55
9.2. Backend (Spring Boot, Java)	56
9.3. Base de datos	56
9.4. Integración con Odoo	56
9.5. DevOps e CI/CD	56
9.6. Distribución iOS	57
10. Anexo I – Manual técnico de instalación ou posta en marcha	57
11. Anexo II – Documentación de uso (manuais usuario)	57
12. Anexo III – Outra documentación	57

1. Introducción

O presente proxecto consiste no desenvolvemento dunha aplicación móbil de preventa deseñada para optimizar a xestión comercial e o proceso de toma de pedidos por parte dos representantes de vendas (preventas). A aplicación permite aos usuarios realizar pedidos de forma eficiente directamente desde os dispositivos móbiles, consultando en tempo real os datos dos clientes existentes na base de datos, o catálogo de produtos e os seus prezos. Tamén incorpora a funcionalidade de dar de alta novos clientes.

A aplicación céntrase en reducir os tempos de xestión, minimizar erros nos pedidos e ofrecer unha ferramenta dixital completa e accesible para mellorar a produtividade.

2. Obxectivos

O desenvolvemento deste proxecto persegue un conxunto de metas ben definidas, tanto a nivel xeral de solución de negocio como a nivel específico de funcionalidades técnicas:

Obxectivo xeral:

- **Optimizar e Dixitalizar o Proceso de Preventa:** Desenvolver unha aplicación móbil que substitúa a xestión manual de pedidos por un proceso dixital eficiente, integrado e en tempo real, mellorando a produtividade do equipo comercial.

Obxectivos específicos:

- **Funcionalidade de Pedidos:** Permitir a creación, modificación e envío de pedidos de forma dixital e segura desde a aplicación móbil, cun tempo medio de procesamento inferior a 5 minutos por pedido.
- **Sincronización en Tempo Real:** Garantir a consulta de datos de clientes, produtos e prezos co backend cun tempo de resposta inferior a 2 segundos (cando haxa conexión de datos).
- **Xestión de Clientes:** Implementar a funcionalidade de alta de novos clientes de forma inmediata, validando os datos esenciais no momento da visita comercial.
- **Traballo Sen Conexión (Offline):** Desenvolver un mecanismo de caché local que permita aos preventas realizar pedidos e consultar datos esenciais mesmo sen conexión a Internet, sincronizando a información automaticamente ao recuperar a conectividade.
- **Usabilidade:** Proporcionar unha interface de usuario (UI) intuitiva e optimizada para o traballo en campo, cunha curva de aprendizaxe mínima para o equipo comercial.

3. Situación previa

Actualmente, o proceso de xestión de pedidos por parte dos representantes de vendas da empresa caracterízase pola súa dependencia de métodos manuais e desactualizados. Os preventas adoitan tomar notas en papel ou utilizar follas de cálculo non conectadas, o que leva a varios problemas operativos:

1. **Ineficiencia e Retrasos:** A necesidade de transcribir os pedidos posteriormente ao sistema central (ERP ou sistema de xestión de almacén) provoca atrasos na tramitación e entrega dos produtos.
2. **Erros na Captura de Datos:** A transcripción manual é propensa a erros, como referencias incorrectas de produtos, prezos desactualizados ou condicións comerciais erróneas, o que xera problemas na facturación e insatisfacción do cliente.
3. **Falta de Información en Tempo Real:** Os preventas non dispoñen de acceso inmediato ao stock actual, ás condicións comerciais específicas do cliente ou á súa historia de pedidos, limitando a súa capacidade de venda consultiva e de resposta rápida.
4. **Dificultade na Xestión de Clientes:** O proceso de alta de novos clientes é lento e require coordinación posterior coa oficina.

Existen alternativas no mercado, principalmente sistemas de Xestión de Relacións con Clientes (CRM) ou Módulos de Vendas de ERPs máis amplos (como SAP ou solucións locais específicas). Non obstante, estas alternativas adoitan ser onerosas na súa implantación, requiren unha curva de aprendizaxe elevada e, en moitos casos, ofrecen unha funcionalidade excesivamente ampla para o obxectivo específico dunha preventa simplificada e enfocada na mobilidade.

O presente proxecto busca ofrecer unha solución áxil, económica e especificamente adaptada aos procesos de preventa no contexto da empresa, sen a complexidade nin o custo dun ERP completo.

4. Tecnoloxías empregadas

Para o desenvolvemento do proxecto, empregárase unha arquitectura moderna e orientada a servizos para garantir a escalabilidade e o rendemento, cos seguintes compoñentes clave:

Frontend:

- **Framework: Flutter** (con **Dart**) empregárase para o desenvolvemento da aplicación móbil, o que permitirá crear unha única base de código para despregamento en múltiples plataformas (Android e/ou iOS).

Backend:

- **Framework:** Utilizarase **Spring Boot** para desenvolver a API RESTful que actuará como o núcleo da lóxica de negocio. Esta API encargárase de xestionar as peticións do frontend, a autenticación e a interacción coa base de datos.
- **Base de datos:** Os datos estruturados do sistema (clientes, produtos, pedidos) almacenaranse nunha base de datos relacional MySQL. Na aplicación móbil, poderase implementar un sistema de caché local para almacenar datos esenciais e permitir o traballo sen conexión.

5. Solución proposta

A solución proposta consiste no desenvolvemento de GuzmanGes, unha aplicación móbil nativa multiplataforma (grazas a Flutter) xunto cunha robusta API RESTful (desenvolvida con Spring Boot) e unha base de datos MySQL centralizada.

Esta arquitectura separa claramente a lóxica de negocio (no backend Spring Boot) da interface de usuario (frontend Flutter), garantindo a escalabilidade, o rendemento e un mantemento sinxelo.

Características diferenciais e valor engadido:

- **Deseño Específico para Preventa:** A diferenza dos módulos de vendas xenéricos de ERPs, GuzmanGes está deseñada exclusivamente para simplificar o proceso de toma de pedidos en campo. A interface e os fluxos de traballo priorizan a velocidade e a mínima interacción.
- **Funcionalidade Offline (Traballo Sen Conexión):** Esta é unha vantaxe clave fronte a moitas solucións baseadas unicamente na web ou en conexión permanente. O sistema de caché local con SQLite na aplicación móbil permite traballar sen conexión, garantindo a continuidade do negocio mesmo en áreas con baixa cobertura de rede, para sincronizar os pedidos tan pronto como se recupere a conectividade.
- **Tecnoloxía Moderna e Eficiente:** A elección de Flutter permite un desenvolvemento rápido e despregamento en Android e iOS cunha única base de código. Spring Boot, coñecido pola súa eficiencia na creación de microservizos e APIs, asegura a dispoñibilidade e rapidez do backend.
- **Integración Sencilla:** A través da API RESTful, o sistema está deseñado para unha futura integración sinxela con outros sistemas internos da empresa (como o sistema de xestión de almacén ou a contabilidade), minimizando o rozamento na adopción.

6. Planificación do proxecto

6.1 Presuposto completo (Hardware, software e man de obra)

A seguinte táboa presenta unha estimación de custos revisada para o desenvolvemento completo do proxecto, asumindo unha duración de 3 meses e unha dedicación de 40 horas semanais, cunhas tarifas de man de obra axustadas a un custo máis económico.

1. Recursos Humanos (Man de Obra)

Rol	Dedicación (Horas)	Custo Hora Estimado (€)	Custo Total (€)
Desenvolvedor Senior/Arquitecto	480	15,00 €	7.200,00 €
Desenvolvedor Junior/Móbil	480	10,00 €	4.800,00 €
Xestor de Proxectos/Análise (25% tempo)	120	20,00 €	2.400,00 €
Subtotal Recursos Humanos	1080		14.400,00 €

2. Software e Licenzas

Para o desenvolvemento, optouse por tecnoloxías de código aberto e gratuítas que minimizan os custos de licenza, excepto o custo do aloxamento.

Descrición	Tipo de Custo	Custo Estimado (€)
Flutter/Dart/Spring Boot/MySQL	Licenza (Open Source)	0,00 €
IDE's de Desenvolvemento (Versións gratuítas)	Licenza	0,00 €
Servidor de Repositorio (GitHub/GitLab)	Servizo (Versións gratuítas)	0,00 €
Servidor de Despregamento (VPS/Cloud)	Custo de subscrición/mes (Estimación)	450,00 €
Subtotal Software e Servizos		450,00 €

3. Hardware

O hardware de desenvolvemento (portátiles, dispositivos de proba) asúmese como xa existente e amortizado pola empresa.

Descrición	Tipo de Custo	Custo Estimado (€)
Equipos de Desenvolvemento e Proba	Custo xa asumido/amortizado	0,00 €
Tablet/Móbil para Empregados (e.g. Samsung/equivalente)	Custo de adquisición/amortización	200,00 €
Subtotal Hardware		200,00 €

4. Orzamento Total do Proxecto

Concepto	Custo Total (€)
Recursos Humanos	14.400,00 €
Software e Servizos	450,00 €
Hardware	200,00 €
TOTAL ORZAMENTO ESTIMADO	15.050,00 €

Nota: Este orzamento é unha estimación baseada en tarifas axustadas. O custo real do servidor de despregamento dependerá do provedor e da carga de traballo esperada. Os prezos indicados anteriormente son sin impostos incluídos.

6.2 Diagrama de Gantt



6.3 Modelo de desenvolvemento de software a implementar

Para o desenvolvemento de GuzmanGes, optouse por unha metodoloxía de desenvolvemento Áxil baseada no marco de traballo Scrum. Esta elección responde á necesidade dun proceso iterativo e incremental que permita a evolución constante tanto da aplicación móbil (Flutter) como da API RESTful (Spring Boot)

1. Xustificación do modelo:

- Desenvolvemento Iterativo e Incremental:** Dado que o proxecto require a integración de múltiples compoñentes (Frontend, Backend e Base de Datos), Scrum permite realizar entregas parciais e funcionais en períodos curtos.
- Xestión de Riscos Técnicos:** A funcionalidade de traballo sen conexión (offline) e o uso de caché local con SQLite son puntos críticos que requiren ciclos frecuentes de proba e axuste, algo que este modelo facilita enormemente.
- Foco na Usabilidade:** Ao ser unha ferramenta para representantes de vendas en campo, a interface debe ser altamente intuitiva e cunha curva de aprendizaxe mínima. O enfoque áxil permite adaptar o deseño de Flutter baseándose no feedback continuo a medida que se constrúen as pantallas.

2. Implementación do proceso (Sprints)

O desenvolvemento estrutúrase en ciclos de traballo denominados **Sprints**, os cales se aliñaron cos fitos de entrega establecidos no calendario oficial do proxecto:

- Sprint 1 (Análise e Deseño):** Definición dos requisitos, deseño do modelo Entidade-Relación, diagrama de casos de uso e creación de *mockups* da interface.
- Sprint 2 (Proba de Concepto):** Configuración das contornas de desenvolvemento (Spring Boot e Flutter), diagrama de despregamento e implementación do primeiro caso de uso (habitualmente a Autenticación).
- Sprint 3 (Implementación de Funcionalidades):** Desenvolvemento do resto do alcance acordado: catálogo de produtos, xestión de clientes, creación de pedidos e a lóxica de sincronización bidireccional *offline/online*.
- Sprint 4 (Peche e Documentación):** Refinado do código, elaboración da memoria de dificultades técnicas, redacción dos manuais (usuario e instalación) e preparación da presentación final.

6.4 Análise do proxecto

6.4.1 Definición do alcance do proxecto

1.- Visión xeral

GuzmanGes é unha aplicación móbil multiplataforma (Android e iOS, desenvolvida con Flutter) destinada aos representantes comerciais (preventas) que visitan clientes en campo. A aplicación permítelles consultar o catálogo de produtos, a cartera de clientes e o histórico de pedidos sincronizados desde o ERP corporativo **Odoo**, así como rexistrar novos pedidos e dar de alta clientes novos durante a propia visita.

A aplicación apóiase nunha **API REST en Spring Boot** que actúa como capa intermedia entre Odoo e o dispositivo móbil, e nunha base de datos **MySQL** que almacena os datos sincronizados desde Odoo e os pedidos novos xerados desde a app.

2. Que fai o proxecto (dentro do alcance)

2.1. Backend (API REST en Spring Boot + MySQL)

- **Autenticación** de usuarios mediante login/contrasinal con tokens JWT.
- **Sincronización con Odoo** (capa Spring Boot ↔ Odoo vía XML-RPC ou JSON-RPC):
 - Importación periódica desde Odoo a MySQL de:
 - Clientes (res.partner).
 - Produtos (product.product).
 - Modos de pagamento e condicións de pagamento.
 - Pedidos do **último ano** (sale.order + sale.order.line).
 - Exportación cara Odoo dos pedidos novos creados desde a aplicación móbil.
 - Exportación cara Odoo dos clientes novos dados de alta desde a aplicación.
- **Endpoints REST** consumidos pola aplicación móbil:
 - Autenticación.
 - Lectura de clientes, produtos, modos e condicións de pagamento, e histórico de pedidos.
 - Creación, modificación, eliminación e envío de pedidos.
 - Creación de clientes novos.
- **Persistencia en MySQL** de toda a información sincronizada e dos novos rexistros xerados desde a app, incluídos os identificadores idOdoo para reconciliación.

2.2. Aplicación móbil (Flutter)

A aplicación está deseñada como un **cliente principalmente de lectura** sobre os datos mestres (clientes, produtos, catálogos), agás polas operacións explícitas que se detallan a continuación.

Operacións de lectura (consulta de datos sincronizados desde Odoo):

- Listaxe e busca de clientes.
- Visualización do detalle dun cliente (datos mestres + histórico de pedidos do último ano, sexan creados na app ou en Odoo).
- Listaxe e busca de produtos.
- Visualización do detalle dun produto.
- Consulta de modos e condicións de pagamento.

Operacións de escritura permitidas dende a app:

- **Alta de cliente** novo con validación dos datos básicos obrigatorios (CIF, razón social, dirección, localidade, código postal, provincia, teléfono).
 - Non se permite editar nin eliminar clientes existentes desde a app: a edición faise en Odoo.
- **CRUD completo de pedidos** mentres estean en estado borrador
 - Crear pedido (cabeceira + liñas con produto, cantidade, prezo, IVE, recargo de equivalencia).
 - Editar pedido borrador (engadir, modificar ou eliminar liñas).
 - Eliminar pedido borrador.
 - **Confirmar/enviar pedido:** cambia o estado a confirmado e queda bloqueado para edición. A partir dese momento o sistema o sincroniza con Odoo.

2.3. Sincronización Flutter ↔ Spring Boot (caché local SQLite)

- A aplicación móbil mantén unha **caché local en SQLite** cos datos mestres (clientes, produtos, catálogos) descargados na última conexión, para permitir o traballo en zonas sen cobertura.
- **Cola de envío** local en SQLite para pedidos creados ou modificados sen conexión, e para clientes novos dados de alta sen conexión.
- Ao recuperar conexión, a app envía a cola á API REST en orde cronolóxica.

- A sincronización é **unidireccional na escritura** (do móbil cara o servidor): non se contempla edición concorrente offline de rexistros existentes.

3. Que NON fai o proxecto (fóra do alcance)

Quedan explicitamente fóra do alcance do MVP as seguintes funcionalidades:

- **Edición ou eliminación de clientes existentes** desde a app móbil (xestiónase en Odoo).
- **Xestión do catálogo de produtos** desde a app (alta, edición, baixa, prezos, stock — todo se xestiona en Odoo).
- **Xestión de modos e condicións de pagamento** desde a app.
- **Edición de pedidos xa confirmados ou xa exportados a Odoo** desde a app (só se permite editar pedidos en estado borrador).
- **Resolución de conflitos en escritura concorrente offline**: non se contempla que dous comerciais editen o mesmo rexistro sen conexión simultaneamente.
- **Xestión de usuarios e permisos** desde a app (alta, baixa, cambio de contrasinal — xestiónanse no backend ou directamente en BD).
- **Xeolocalización** dos comerciais ou rutas de visitas.
- **Xeración de reportes**, estatísticas, gráficas ou cadros de mando.
- **Xestión de pagamentos** ou cobros desde a app.
- **Soporte multi-empresa ou multi-divisa**.
- **Notificacións push** (avisos de novos clientes, alertas de stock, etc.).
- **Integración con sistemas externos distintos de Odoo** (CRMs, plataformas de e-commerce, contabilidade externa, etc.).

4. Sistemas externos

4.1. Odoo (ERP)

Aspecto	Descrición
Forma parte do proxecto?	Si , a integración con Odoo está dentro do alcance.
Rol	Sistema de información mestre para clientes, produtos, catálogos de pagamento e pedidos da empresa.
Versión asumida	Odoo Community Edition (autoinstalado en Docker)
Protocolo de integración	XML-RPC ou JSON-RPC (API estándar de Odoo). A elección concreta documentarase na memoria de execución.
Modelos de Odoo implicados	res.partner (clientes), product.product (produtos), account.payment.term (condicións de pagamento), sale.order e sale.order.line (pedidos).
Dirección da sincronización	Bidireccional: Odoo → MySQL (lectura periódica)

	de mestres e pedidos do último ano) e MySQL → Odoo (envío de pedidos novos e clientes novos).
Reconciliación	Cada entidade sincronizable na BD ten un campo idOdoo que almacena o identificador en Odoo. Os rexistros creados desde a app reciben o seu idOdoo despois da subida exitosa a Odoo.
Frecuencia de sync	Sincronización dende Odoo programada periodicamente (xob no backend) ou disparada manualmente desde a app mediante o botón "actualizar". A subida de pedidos faise inmediatamente tras a confirmación cando hai conexión, ou desde a cola de SQLite ao recuperala.

4.2. Outros sistemas

- **Servidor de despregamento (VPS / Cloud):** infraestrutura externa onde se hospeda a API Spring Boot e a base de datos MySQL. Non forma parte do desenvolvemento, pero si do despregamento.
- **Repositorio de código (GitHub):** servizo externo para o control de versións. Non forma parte do desenvolvemento funcional.

5. Restricións e supostos

Restricións técnicas

- A aplicación móbil debe funcionar en **Android e iOS** desde unha única base de código (Flutter).
- O backend está restrinxido ao stack acordado: **Spring Boot 4 + Java 21 + MySQL**.
- O tempo de resposta dos endpoints de lectura debe ser inferior a **2 segundos** cando hai conexión.
- O tempo medio de procesamento dun pedido completo (creación + confirmación) debe ser inferior a **5 minutos**.

Supostos de negocio

- Cada comercial ten asignada unha cartera de clientes (relación CLIENTE.comercial_id).
- O catálogo de produtos é común a toda a empresa.
- Os prezos de venda son os definidos en Odoo. Non se contempla negociación de prezos no momento da venda desde a app (o prezo de cada liña é o do produto en Odoo no momento da última sincronización).
- O comercial nunca elimina rexistros de Odoo desde a app: as eliminacións (clientes, produtos) realízanse exclusivamente en Odoo.

Supostos de despregamento

- Existe unha instancia de Odoo Community Edition accesible desde o backend Spring Boot.
- O backend está hospedado nun servidor con conectividade permanente a internet.
- O dispositivo móbil ten conectividade intermitente (4G/5G/WiFi) e debe poder operar nas xanelas sen conexión.

6.4.2 Universo de discurso da base de datos

1. Contexto

A empresa para a que se desenvolve **GuzmanGes** dedícase á distribución de produtos e mantén un equipo de **representantes comerciais (preventas)** que visitan clientes en campo para tomar pedidos. Toda a información de clientes, produtos e pedidos histórica vive no sistema de planificación de recursos da empresa, **Odoo**, que actúa como sistema mestre.

A base de datos relacional **MySQL** que sustenta a API Spring Boot replica a información necesaria de Odoo e almacena os pedidos novos xerados desde a aplicación móbil mentres non son enviados a Odoo, así como os pedidos confirmados xa sincronizados.

2. Descrición das entidades

2.1. Usuario

A empresa dispón dun **conxunto de usuarios** que poden iniciar sesión na aplicación móbil. Cada usuario representa un comercial (preventa) ou un perfil administrativo. De cada usuario almacénanse os seus datos persoais e de acceso: nome completo, nome de usuario único para o login, enderezo de correo electrónico, contrasinal cifrado e un tipo de usuario que identifica o seu rol (por exemplo, comercial ou administrador).

Os usuarios non se sincronizan con Odoo: créanse e mantéñense exclusivamente no sistema GuzmanGes.

2.2. Cliente

Un **cliente** representa un local, comercio ou empresa á que se lle venden produtos. De cada cliente almacénanse:

- Identificador interno (xerado automaticamente).
- Identificador en Odoo (idOdoo): permite reconciliar o rexistro co sistema mestre.
- Nome comercial e razón social.
- CIF (identificación fiscal).
- Dirección postal completa: rúa, localidade, código postal e provincia.
- Datos de contacto: teléfono fixo, móbil, correo electrónico.
- Posición fiscal (réxime fiscal aplicable: por exemplo, réxime xeral, recargo de equivalencia).
- Marca de actividade (activo) e data da última modificación.

Cada cliente ten asignados de xeito obrigatorio:

- Un **modo de pagamento** (forma na que paga: transferencia, tarxeta, contado, etc.).
- Unha **condición de pagamento** (prazos: contado, 30 días, 60 días, 90 días, etc.).
- Un **comercial** responsable do cliente, é dicir, o usuario que xestiona habitualmente esa conta.

Os clientes provéñen maioritariamente de Odoo (sincronización descendente), aínda que o sistema permite dar de alta clientes novos desde a aplicación móbil. Eses novos clientes envíanse a Odoo na seguinte sincronización ascendente, momento no que reciben o seu idOdoo definitivo.

2.3. Modo de pagamento

Catálogo simple xestionado en Odoo. Cada **modo de pagamento** ten un identificador interno, un identificador en Odoo (idOdoo), unha descrición textual (por exemplo, transferencia bancaria, efectivo, cheque), unha marca de actividade e unha data da última modificación. Sincronízase periodicamente desde Odoo a MySQL e é só de lectura na aplicación móbil.

2.4. Condición de pagamento

Catálogo simple xestionado en Odoo. Cada **condición de pagamento** ten un identificador interno, un identificador en Odoo (idOdoo), unha descrición textual (por exemplo, contado, 30 días, 60 días fin de mes), unha marca de actividade e unha data da última modificación. Sincronízase periodicamente desde Odoo a MySQL e é só de lectura na aplicación móbil.

2.5. Produto

Un **produto** representa un ítem do catálogo de venda da empresa. De cada produto almacénanse:

- Identificador interno e identificador en Odoo (idOdoo).
- Referencia interna do produto (código curto usado pola empresa).
- Código de barras.
- Descrición.
- Tipo de produto (por exemplo, almacenable, consumible, servizo).
- Stock dispoñible (cantidade en almacén segundo a última sincronización).
- Prezo de venda.
- Observacións ou notas internas.
- Data da última modificación.

O catálogo de produtos é mestre en Odoo: sincronízase periodicamente cara MySQL e é só de lectura na aplicación móbil. Non se xestiona alta, modificación nin baixa de produtos desde GuzmanGes.

2.6. Pedido

Un **pedido** é unha orde de compra realizada por un cliente nunha visita do comercial (ou ben rexistrada directamente en Odoo desde a oficina). De cada pedido almacénanse:

- Identificador interno e identificador en Odoo (idOdoo, que estará baleiro mentres o pedido aínda non se enviou a Odoo).
- Número de pedido (referencia visible).
- Data de creación.
- Importes calculados: base imponible (totalBase), total de IVE (totalIva), total de recargo de equivalencia (totalRE) e total final (total).
- **Estado do pedido:** describe a fase comercial do pedido. Os valores posibles son:
 - borrador: pedido en construción, modificable.
 - confirmado: pedido pechado polo comercial, listo para enviar a Odoo.
 - anulado: pedido cancelado.
- **Estado de sincronización:** describe se o pedido foi enviado correctamente a Odoo. Os valores posibles son:
 - pendente: aínda non enviado a Odoo.
 - sincronizado: enviado correctamente, ten idOdoo.
 - erro: a subida fallou e debe reintentarse.
- Data da última modificación.

Cada pedido pertence sempre a:

- Un único **cliente**.
- Un único **usuario** (o comercial que rexistrou o pedido). No caso de pedidos importados desde Odoo que non foran feitos por ningún comercial da app, asígnaselles un usuario administrativo predeterminado.

Un pedido contén unha ou varias **liñas de pedido** que detallan os produtos solicitados.

2.7. Liña de pedido

Unha **liña de pedido** representa cada produto incluído nun pedido coa súa cantidade e prezo. De cada liña almacénanse:

- Identificador interno.
- Código do produto (snapshot da referencia no momento do pedido).
- Descrición do produto (snapshot).
- Prezo unitario aplicado.
- Porcentaxe de IVE.
- Porcentaxe de recargo de equivalencia (no caso de clientes con esa posición fiscal).
- Subtotal da liña.

Cada liña pertence a un único **pedido** e referencia un único **produto**. As liñas non existen de xeito independente: cando se elimina un pedido borrador, elimínanse tamén as súas liñas.

Almacénanse o código e a descrición do produto **na propia liña** (en lugar de obter os datos do produto referenciado) para conservar a información histórica do pedido aínda que o produto orixinal se modifique ou se elimine en Odoo posteriormente.

3. Relacións entre entidades

A continuación descríbense as relacións do dominio coas súas cardinalidades.

3.1. Usuario — Cliente (xestiona)

Cada **cliente** está asignado a un **comercial** (usuario) que é o responsable habitual da conta. Un mesmo usuario pode xestionar moitos clientes (a súa carteira), e cada cliente ten exactamente un comercial asignado.

- Un Usuario xestiona **0..N** Clientes.
- Un Cliente é xestionado por **1** Usuario.

3.2. Cliente — Modo de pagamento (paga con)

Cada **cliente** ten asociado un único **modo de pagamento** habitual. Un mesmo modo de pagamento pode estar asignado a moitos clientes.

- Un Cliente paga con **1** Modo de pagamento.
- Un Modo de pagamento asígnase a **0..N** Clientes.

3.3. Cliente — Condición de pagamento (condición)

Cada **cliente** ten asociada unha única **condición de pagamento** habitual. Unha mesma condición pode estar asignada a moitos clientes.

- Un Cliente ten **1** Condición de pagamento.

- Unha Condición de pagamento asígnase a **0..N** Clientes.

3.4. Cliente — Pedido (realiza)

Un **cliente** pode ter múltiples pedidos ao longo do tempo. Cada pedido pertence sempre a un único cliente.

- Un Cliente realiza **0..N** Pedidos.
- Un Pedido realízase para **1** Cliente.

3.5. Usuario — Pedido (rexistra)

Cada **pedido** foi rexistrado por un único **usuario** (o comercial que o tomou en visita ou un usuario administrativo no caso de pedidos importados desde Odoo). Un usuario pode ter rexistrado moitos pedidos.

- Un Usuario rexistra **0..N** Pedidos.
- Un Pedido é rexistrado por **1** Usuario.

3.6. Pedido — Liña de pedido (contén)

Un **pedido** contén unha ou varias **liñas**, e cada liña pertence sempre a un único pedido. As liñas son entidades **débiles**: non existen sen o pedido pai e elimínanse en cascada se o pedido se elimina.

- Un Pedido contén **1..N** Liñas de pedido.
- Unha Liña de pedido pertence a **1** Pedido.

3.7. Produto — Liña de pedido (referencia)

Cada **liña de pedido** referencia un único **produto** do catálogo. Un mesmo produto pode aparecer en moitas liñas de moitos pedidos distintos.

- Un Produto aparece en **0..N** Liñas de pedido.
- Unha Liña de pedido referencia **1** Produto.

4. Restricións e regras de negocio

A continuación recóllense as principais restricións de integridade e regras de negocio que se reflectirán no modelo de datos:

1. **Integridade referencial:** as eliminacións de clientes, produtos, modos e condicións de pagamento están restrinxidas: non se pode eliminar un rexistro mestre se hai pedidos ou clientes que o referencian.
2. **Eliminación en cascada:** ao eliminar un pedido en estado borrador, elimínanse automaticamente as súas liñas.
3. **Inmutabilidade de pedidos confirmados:** un pedido en estado confirmado ou anulado non se pode editar nin eliminar desde a aplicación.
4. **Reconciliación con Odoo:**
 - a. Os rexistros descargados de Odoo teñen sempre idOdoo definido.
 - b. Os rexistros creados desde a aplicación nacen con idOdoo baleiro e enchense unha vez sincronizados con éxito.
 - c. O par (idOdoo, tipo de entidade) é único cando idOdoo non é nulo.
5. **Xanela temporal de sincronización:** desde Odoo só se importan os pedidos do **último ano** respecto á data actual. Os pedidos máis antigos permanecen exclusivamente en Odoo.
6. **Identificadores de usuario:** o nomeUsuario do usuario é único en todo o sistema.

7. **Datos obrigatorios na alta de cliente desde a app:** ao crear un cliente desde a aplicación móbil, son obrigatorios CIF, razón social, dirección, localidade, código postal, provincia, modo de pagamento e condición de pagamento.
8. **Coherencia de totais:** o total dun pedido debe ser igual á suma de subtotais das súas liñas máis os impostos correspondentes ($\text{totalBase} + \text{totalIva} + \text{totalRE}$). Esta coherencia garante a lóxica do backend ao gardar o pedido.
9. **Consistencia entre estado e idOdo:** un pedido con `estadoSync = sincronizado` debe ter `idOdo` definido. Un pedido con `estadoSync = pendente` ou erro pode ter `idOdo` baleiro (se nunca chegou a Odo) ou definido (se sincronizou nalgún momento e logo se modificou).

5. Resumo das entidades e cardinalidades

Entidade	Tipo	Sincronizable con Odoo	Editable desde a app
Usuario	Forte	Non	Non (xestionado en BD)
Cliente	Forte	Si (bidireccional)	Só alta
Modo de pagamento	Forte (catálogo)	Si (descendente)	Non
Condición de pagamento	Forte (catálogo)	Si (descendente)	Non
Produto	Forte (catálogo)	Si (descendente)	Non
Pedido	Forte	Si (bidireccional)	CRUD completo en estado borrador
Liña de pedido	Débil (depende de Pedido)	Si (xunto co Pedido)	CRUD coa cabeceira borrador

Relación	Cardinalidade
Usuario xestiona Cliente	1 : 0..N
Cliente paga con Modo de pagamento	N : 1
Cliente ten Condición de pagamento	N : 1
Cliente realiza Pedido	1 : 0..N
Usuario rexistra Pedido	1 : 0..N
Pedido contén Liña de pedido	1 : 1..N
Produto referenciado por Liña de pedido	1 : 0..N

6.4.3 Diagramas Base de Datos

Diagrama Entidad-Relación con notación Chen:

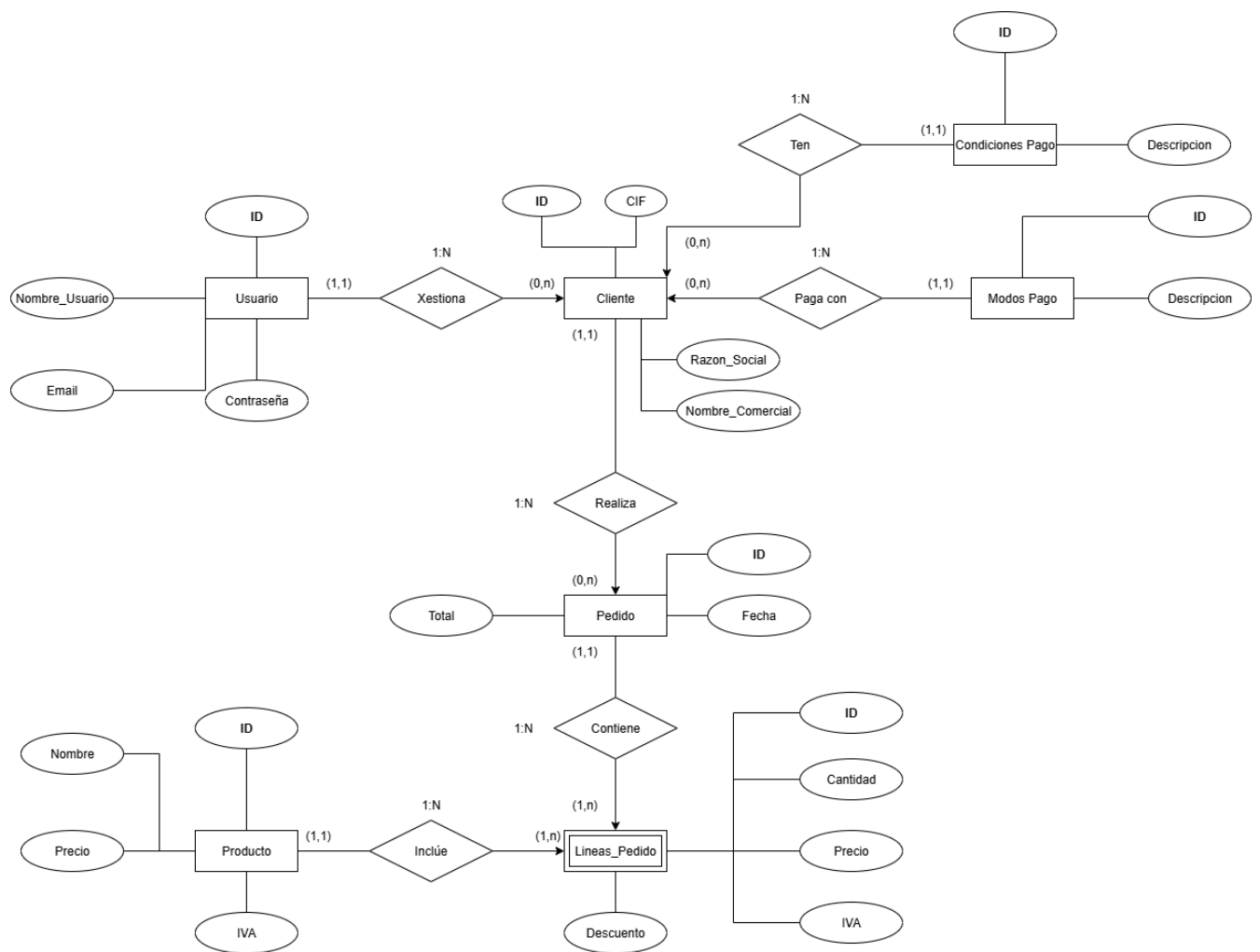
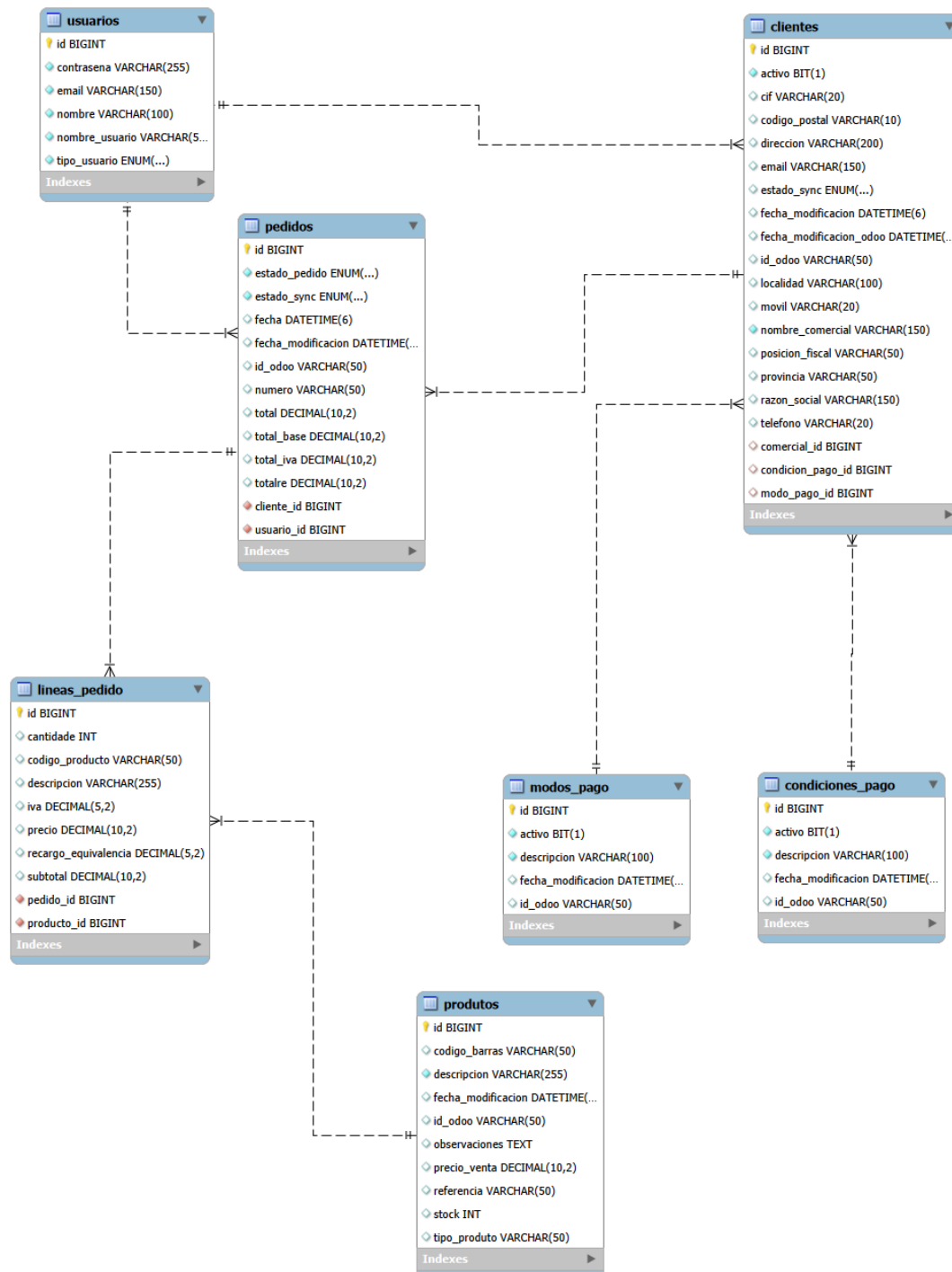
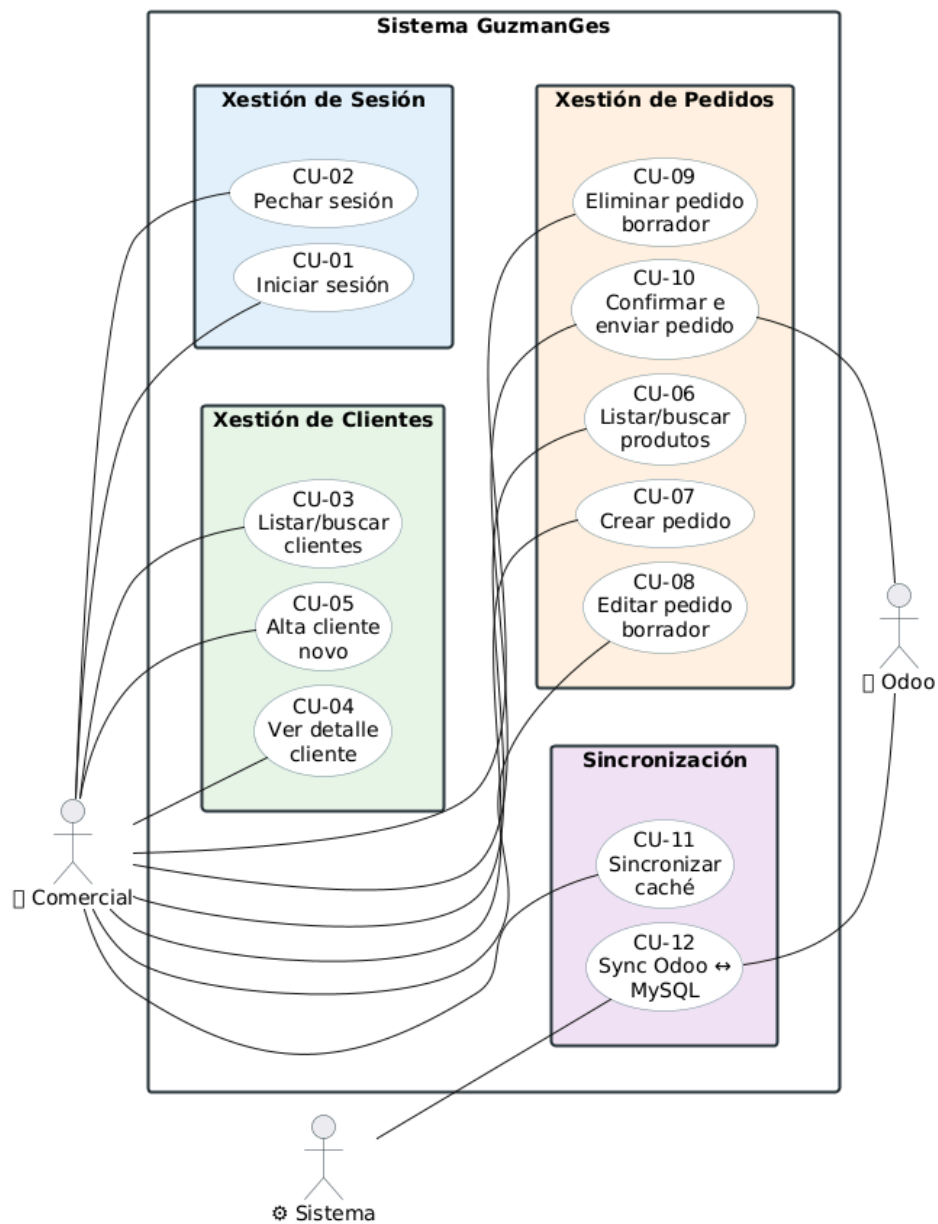


Diagrama modelo relacional con notacion Crow's Foot:



6.4.4 Diagrama de casos de uso



6.4.5 Descrición dos casos de uso

CU-01	Iniciar sesión		
Descrición	O comercial accede á aplicación introducindo as súas credenciais para obter unha sesión autenticada cun token JWT.		
Actores	Comercial.		
Pre condicións	• A aplicación está instalada no dispositivo. • O usuario está dado de alta no sistema (creado polo administrador). • Hai conexión a Internet (necesaria a primeira vez).		
Post condicións	• O comercial ten unha sesión activa. • O token JWT está almacenado localmente. • A aplicación mostra a pantalla principal.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	Abre a aplicación.	Mostra a pantalla de login.
	2	Introduce nome de usuario e contrasinal.	Valida o formato dos campos en tempo real.
	3	Preme "Entrar".	Envía as credenciais ao endpoint POST /auth/login.
	4		O backend valida contra a táboa USUARIO, xera un token JWT e devólveo.
	5		A aplicación garda o token localmente e navega á pantalla principal.
Excepciones	#	Acción (actor)	Reacción (sistema)
	2a	Algún campo baleiro ou con formato inválido.	Mostra mensaxe de validación local sen enviar a petición.
	3a	Credenciais incorrectas (backend responde HTTP 401).	Mostra "Usuario ou contrasinal incorrectos" e mantén o comercial na pantalla de login.
	3b	Sen conexión.	Mostra "Necesítase conexión para iniciar sesión por primeira vez. Comproba a túa rede."

	3c	Erro do servidor (HTTP 5xx).	Mostra "Erro temporal no servidor. Inténtao de novo en uns minutos."
Rendimiento	O sistema deberá completar a autenticación en menos de 2 segundos con conexión		
Frecuencia	1-2 veces ao día por comercial (ao comezar a xornada).		
Importancia	Vital		
Urgencia	Inmediatamente		
Comentarios	O token JWT terá unha expiración configurable (ex: 8 horas). Se caduca durante o uso, a app detectará HTTP 401 nas peticións e redirixirá ao login		

CU-02	Pechar sesión		
Descripción	O comercial finaliza a súa sesión, eliminando o token JWT local.		
Actores	Comercial		
Pre condiciones	• O comercial ten unha sesión activa.		
Post condiciones	• O token JWT está eliminado do almacenamento local. • A aplicación mostra a pantalla de login. • A caché local SQLite non é accesible ata reiniciar sesión.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	Preme "Pechar sesión" no menú.	Mostra diálogo de confirmación.
	2	Confirma.	Elimina o token JWT do almacenamento local.
	3		Navega á pantalla de login.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1a	Hai pedidos pendentes na cola de envío.	Avisa: "Hai N pedidos pendentes de sincronizar. Se pechas sesión perderanse. ¿Continuar?". Se o comercial confirma, pérdese a referencia á cola na sesión actual.
	2a	O comercial cancela.	Permanece na pantalla actual.
Rendimiento	Acción completa en menos de 1 segundo.		
Frecuencia	1-2 veces ao día por comercial.		
Importancia	Importante		
Urgencia	Pode esperar.		
Comentarios	A acción é exclusivamente local; non require conexión nin notifica ao backend. O token segue válido ata caducar, pero xa non é accesible para a app.		

CU-03	Listar e buscar clientes		
Descripción	O comercial consulta a listaxe da súa carteira de clientes, podendo filtrala por nome ou CIF.		
Actores	Comercial.		
Pre condiciones	• O comercial ten unha sesión activa. • Existe polo menos unha sincronización inicial de clientes na caché local (ou o backend está accesible).		
Post condiciones	• O comercial visualiza a listaxe filtrada (ou completa) de clientes. • Ningún dato modificado.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	Accede á sección "Clientes".	Le os clientes da caché SQLite local (filtrados por comercial_id) e móstraos en lista paxinada.
	2	Escribe texto no campo de busca.	Filtra en tempo real por nome comercial, razón social ou CIF.
	3		Mostra os resultados filtrados.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1a	Caché local baleira (primeira vez sen sincronizar).	Mostra "Non hai clientes na caché. Sincroniza para descargar a túa carteira" e ofrece botón que lanza CU-11.
Rendimiento	A busca filtra resultados en menos de 500 ms para listaxes de ata 1.000 clientes.		
Frecuencia	20–30 veces ao día por comercial.		
Importancia	Vital		
Urgencia	Inmediatamente		
Comentarios	A lista mostra só os clientes asignados ao comercial actual (comercial_id = usuario actual). Funciona 100 % offline desde a caché.		

CU-04	Ver detalle de cliente		
Descripción	O comercial consulta a información completa dun cliente, incluíndo os seus datos mestres e o histórico de pedidos do último ano.		
Actores	Comercial.		
Pre condiciones	• O comercial ten unha sesión activa. • O cliente seleccionado existe na caché local.		
Post condiciones	• O comercial visualiza os datos completos do cliente e o seu histórico. • Ningún dato modificado.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	Selecciona un cliente desde a listaxe (precondición de CU-03).	Le os datos do cliente da caché SQLite e o seu histórico de pedidos do último ano.
	2		Mostra unha pantalla con datos do cliente arriba e a listaxe de pedidos abaixo, ordenados de máis recente a máis antigo (data, número, total, estado).
	3	Pulsa nun pedido do histórico.	Navega ao detalle dese pedido (só lectura se non está en borrador).
	4	Pulsa "Novo pedido para este cliente".	Navega a CU-07 cos datos do cliente xa preseleccionados.
Excepciones	#	Acción (actor)	Reacción (sistema)
	2a	Histórico baleiro.	Mostra "Sen pedidos no último ano."
	2b	Algún pedido ten estadoSync = erro.	Mostra unha advertencia ao seu lado na lista para alertar ao comercial.
Rendimiento	A pantalla cárgase en menos de 1 segundo.		

Frecuencia	15-25 veces ao día por comercial.
Importancia	Importante
Urgencia	Inmediatamente
Comentarios	O atallo "Novo pedido para este cliente" mellora a velocidade do fluxo principal da app.

CU-05	Dar de alta cliente novo		
Descripción	O comercial rexistra un cliente novo captado nunha visita, validando os datos básicos obrigatorios.		
Actores	Comercial.		
Pre condiciones	• O comercial ten unha sesión activa.		
Post condiciones	• O cliente novo está rexistrado na BD local (SQLite) con idOdoos = null. • Engadido á cola de envío para subir a Odoos na próxima sincronización. • Asignado ao comercial que o creou (comercial_id = usuario actual)		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	Preme "Novo cliente" na pantalla de listaxe.	Mostra o formulario de alta con campos baleiros.
	2	Enche os campos obrigatorios (CIF, razón social, dirección, localidade, código postal, provincia, teléfono, modo de pagamento, condición de pagamento) e os opcionais que desexe.	Valida cada campo en tempo real (formato CIF, código postal numérico, email, etc.).
	3	Preme "Gardar".	Valida o conxunto de campos obrigatorios.
	4		Garda o cliente en SQLite con idOdoos = null e estado "pendente de subir a Odoos".
	5		Se hai conexión, envía o cliente ao backend mediante POST /clientes. Se non, queda na cola SQLite.
	6		Mostra "Cliente creado correctamente" e navega ao detalle do novo cliente.
Excepciones	#	Acción (actor)	Reacción (sistema)

	3a	Datos inválidos (algún campo obrigatorio baleiro ou con formato incorrecto).	Marca os campos problemáticos en vermello, mostra mensaxe baixo cada un e non envía a petición.
	4a	CIF duplicado na carteira local.	Mostra "Xa existe un cliente con este CIF na túa carteira" e non garda.
	5a	Sen conexión.	Garda localmente e engade á cola; mostra "Cliente gardado. Sincronizarase cando recuperes conexión."
	5b	Erro do backend (HTTP 5xx).	Mantén na cola e mostra "Cliente gardado. Inténtao sincronizar máis tarde."
Rendimiento	Alta completa en menos de 2 segundos con conexión; instantáneo sen conexión.		
Frecuencia	1-3 veces por semana por comercial.		
Importancia	Importante		
Urgencia	Hai presión (pode haber clientes esperando confirmar pedido no momento).		
Comentarios	Tras a sincronización exitosa con Odoo, o backend devolve o idOdoo xerado e a BD local actualízase. Os modos e condicións de pagamento son listas pechadas que veñen sincronizadas desde Odoo (non editables desde a app).		

CU-06	Listar e buscar produtos		
Descrición	O comercial consulta o catálogo de produtos, podendo filtralo por nome, referencia ou código de barras.		
Actores	Comercial.		
Pre condicións	• O comercial ten unha sesión activa. • Existe polo menos unha sincronización inicial de produtos na caché local.		
Post condicións	• O comercial visualiza a listaxe (filtrada ou non) de produtos. • Ningún dato modificado.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	Accede á sección "Produtos".	Le os produtos da caché SQLite local e móstraos en lista.
	2	Escribe texto no campo de busca.	Filtra en tempo real por referencia, descrición ou código de barras.
	3		Mostra os resultados (referencia, descrición, prezo, stock dispoñible).
	4	Selecciona un produto.	Mostra o detalle (descrición ampliada, observacións, prezo, stock).
Excepciones	#	Acción (actor)	Reacción (sistema)
	1a	Caché local baleira.	Mostra "Non hai produtos na caché. Sincroniza para descargarlos" e ofrece botón que lanza CU-11.
Rendimiento	Busca en menos de 500 ms para catálogos de ata 5.000 produtos.		
Frecuencia	30–50 veces ao día por comercial (uso constante durante a creación de pedidos).		
Importancia	Vital		
Urgencia	Inmediatamente		
Comentarios	A pantalla destaca visualmente os produtos con stock = 0 (informativo, non bloquea a venda — a xestión real do stock está en Odoo).		

CU-07	Crear pedido		
Descripción	O comercial crea un pedido novo para un cliente, engadindo liñas con produtos e cantidades. Este é o caso de uso central da aplicación.		
Actores	Comercial.		
Pre condiciones	• O comercial ten unha sesión activa. • Existe polo menos un cliente na caché local. • Existe polo menos un produto na caché local.		
Post condiciones	• O pedido créase na BD local con estadoPedido = borrador, estadoSync = pendente, idOdo = null. • Contén polo menos unha liña. • Os totais (totalBase, totalIva, totalRE, total) calculáronse automaticamente.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	Preme "Novo pedido".	Mostra o formulario de cabeceira e solicita seleccionar cliente.
	2	Selecciona un cliente (busca ou listaxe).	Enche automaticamente os datos fiscais e a condición de pagamento do pedido coa información do cliente.
	3	Preme "Engadir liña".	Mostra a pantalla de selección de produto.
	4	Selecciona un produto e introduce a cantidade.	Calcula automaticamente o subtotal da liña (prezo × cantidade) aplicando IVE e recargo de equivalencia segundo a posición fiscal do cliente.
	5	Repite 3-4 para cada produto adicional.	Engade as liñas e recalcula os totais do pedido.
	6	Pode modificar cantidades ou eliminar liñas.	Recalcula os totais en tempo real.
	7	Preme "Gardar como borrador".	Valida que haxa cliente e polo menos unha liña.

	8		Garda o pedido en SQLite con estadoPedido = borrador, estadoSync = pendente.
	9		Mostra "Pedido gardado como borrador" e navega á listaxe de pedidos.
Excepciones	#	Acción (actor)	Reacción (sistema)
	4a	Cantidade inválida (0, negativa, non numérica).	Mostra "Cantidade inválida" baixo o campo e impide engadir a liña.
	7a	Pedido sen cliente.	Mostra "Selecciona un cliente" e mantén o formulario aberto.
	7b	Pedido sen liñas.	Mostra "O pedido debe ter polo menos unha liña" e mantén o formulario aberto.
Rendimiento	O gardado complétase instantaneamente (operación 100 % local).		
Frecuencia	8–15 veces ao día por comercial.		
Importancia	Vital		
Urgencia	Inmediatamente		
Comentarios	Funciona 100 % offline. O envío a Odoo ocorre en CU-10 cando o comercial confirma o pedido. Pódese acceder tamén desde CU-04 ("Novo pedido para este cliente").		

CU-08	Editar pedido borrador		
Descripción	O comercial modifica un pedido existente en estado borrador, engadindo, eliminando ou alterando liñas.		
Actores	Comercial.		
Pre condiciones	• O comercial ten unha sesión activa. • Existe un pedido en estado borrador na BD local. • O pedido pertence ao comercial (creado por el).		
Post condiciones	• O pedido actualízouse na BD local. • Os totais recalculáronse automaticamente. • O fechaModificacion actualízouse.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	Selecciona un pedido en estado borrador desde a listaxe.	Mostra o formulario de edición con todos os datos cargados.
	2	Modifica os campos desexados (engadir/eliminar liñas, cambiar cantidades, cambiar cliente se aínda non hai liñas).	Recalcula totais en tempo real.
	3	Preme "Gardar cambios".	Valida (mesmas regras que CU-07 paso 7).
	4		Garda en SQLite e actualiza fechaModificacion.
	5		Mostra "Pedido actualizado" e navega ao detalle.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1a	Pedido non está en estado borrador.	Mostra "Este pedido xa está confirmado e non se pode editar." e abre o pedido en modo só lectura.
	3a	Pedido queda sen liñas tras a edición.	Avisa "O pedido debe ter polo menos unha liña". Ofrece "Eliminar pedido" (lanza CU-09).

Rendimiento	o gardado complétase instantaneamente.
Frecuencia	3–5 veces ao día por comercial.
Importancia	Importante
Urgencia	Hai presión.
Comentarios	Cambiar de cliente só está permitido se o pedido aínda non ten liñas, dado que as condicións fiscais (IVE, recargo equivalencia) dependen do cliente e cambialo invalidaría os cálculos.

CU-09	Eliminar pedido borrador		
Descripción	O comercial descarta un pedido en estado borrador que xa non vai enviar.		
Actores	Comercial.		
Pre condiciones	• O comercial ten unha sesión activa. • Existe un pedido en estado borrador na BD local. • O pedido pertence ao comercial.		
Post condiciones	• O pedido elimínase da BD local xunto coas súas liñas (eliminación en cascada por composición). • Non hai impacto en Odoo (o pedido nunca chegou a sincronizarse).		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	Selecciona un pedido en estado borrador.	Mostra o detalle.
	2	Preme "Eliminar pedido".	Mostra diálogo "¿Seguro que queres eliminar este pedido borrador? Esta acción non se pode desfacer."
	3	Confirma.	Elimina o rexistro de PEDIDO (cascada sobre LINEAS_PEDIDO).
	4		Mostra "Pedido eliminado" e navega á listaxe.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1a	Pedido non está en borrador.	O botón "Eliminar" non aparece na pantalla (acción non permitida).
	2a	O comercial cancela.	Permanece no detalle.
Rendimiento	O borrado complétase instantaneamente.		
Frecuencia	1-2 veces por semana por comercial (excepcional).		

Importancia	Quedaría ben.
Urgencia	Pode esperar.
Comentarios	A eliminación é exclusivamente local. Os pedidos confirmados ou sincronizados con Odoo non se poden eliminar desde a app — esa xestión faríase directamente en Odoo.

CU-10	Confirmar e enviar pedido		
Descrición	O comercial confirma un pedido en estado borrador, pasándoo a confirmado e disparando o envío a Odoo.		
Actores	Comercial (primario), Odoo (secundario).		
Pre condicións	• O comercial ten unha sesión activa. • Existe un pedido en estado borrador na BD local. • O pedido ten polo menos unha liña e todos os datos obrigatorios.		
Post condicións	• estadoPedido = confirmado e o pedido xa non é editable nin eliminable desde a app. • Con conexión e resposta correcta de Odoo: estadoSync = sincronizado e idOdoo definido. • Sen conexión ou con erro: estadoSync = pendente ou erro, queda na cola para reintentar.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	Visualiza un pedido en estado borrador e preme "Confirmar pedido".	Mostra diálogo "Tras confirmar, o pedido enviarase a Odoo e xa non poderás editalo. ¿Continuar?"
	2	Confirma.	Actualiza estadoPedido = confirmado e fechaModificacion en SQLite.
	3		Engade o pedido á cola de envío con estadoSync = pendente.
	4		Se hai conexión, envíao ao backend mediante POST /pedidos.
	5		O backend persísteo en MySQL e chama a Odoo vía XML-RPC para crear o sale.order.
	6		Odoo devolve o id do pedido creado.
	7		O backend actualiza o idOdoo en MySQL e responde con éxito ao móbil.
	8		A app actualiza estadoSync = sincronizado e idOdoo en SQLite.

	9		Mostra "Pedido enviado correctamente. Referencia Odoo: [idOdoo]" e navega á listaxe.
Excepciones	#	Acción (actor)	Reacción (sistema)
	4a	Sen conexión.	Marca estadoSync = pendente, mantén na cola e mostra "Pedido confirmado. Enviarase a Odoo cando recuperes conexión."
	5a	Erro do backend (HTTP 5xx).	Marca estadoSync = erro, mostra "Erro ao enviar a Odoo. Pode reintentarse manualmente desde a listaxe." e mantén na cola.
	6a	Odoo rexeita o pedido (cliente inexistente en Odoo, produto inexistente, validación fallida — HTTP 422).	Marca estadoSync = erro e mostra a descrición devolto por Odoo para que o comercial saiba que facer.
Rendimiento	Confirmación e envío en menos de 5 segundos con conexión (incluíndo Odoo); menos de 1 segundo sen conexión (queda só local).		
Frecuencia	8–15 veces ao día por comercial (un envío por cada pedido finalizado).		
Importancia	Vital		
Urgencia	Inmediatamente		
Comentarios	Materializa o valor da integración con Odoo. Pedidos en estadoSync = erro pódense reintentar manualmente desde un botón "Reintentar envío" no detalle. Os clientes novos referenciados sincronízanse primeiro automaticamente para garantir integridade referencial en Odoo.		

CU-11	Sincronizar caché desde o servidor		
Descrición	O comercial dispara manualmente unha actualización da caché local descargando do backend os datos máis recentes de clientes, produtos, catálogos e pedidos.		
Actores	Comercial.		
Pre condicións	• O comercial ten unha sesión activa. • Hai conexión a Internet.		
Post condicións	• A caché local SQLite contén os datos máis actualizados do backend. • A cola de envío (se houbera elementos) intentouse procesar.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	Preme o botón "Sincronizar" no menú.	Mostra indicador de progreso e desactiva interactividade.
	2	[siguiente paso que hace actor]	Envía primeiro a cola de pedidos e clientes pendentes ao backend (subida).
	3		Descarga: clientes da súa carteira, catálogo de produtos, modos e condicións de pagamento, pedidos do último ano (baixada).
	4		Actualiza SQLite cos datos recibidos (insertar/actualizar segundo fechaModificacion).
	5		Mostra "Sincronización completa. N rexistros actualizados." e oculta o indicador.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1a	Hai unha sincronización en curso noutra acción.	O botón aparece desactivado; mensaxe "Sincronizando...".
	2a	Sen conexión.	Mostra "Non hai conexión. Comproba a túa rede." e cancela.

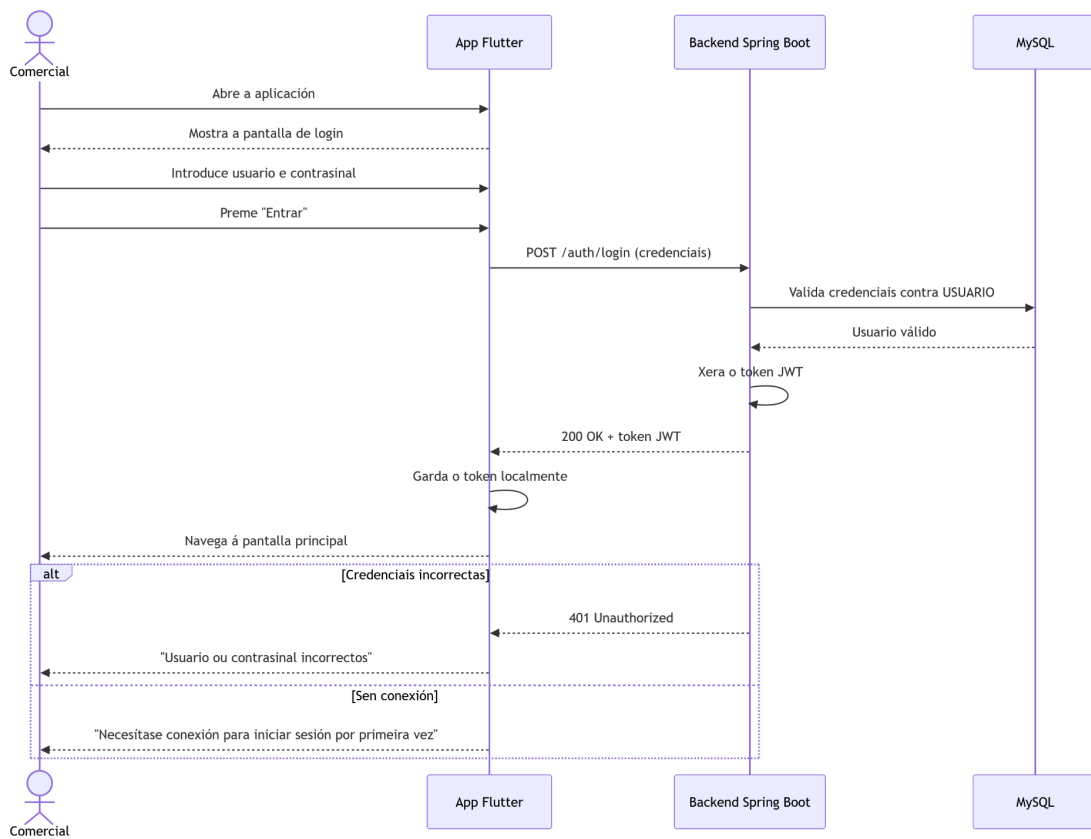
	2b	Erro ao subir un elemento da cola (pedido rexeitado por Odoo, etc.).	Marca ese elemento como estadoSync = erro e continúa coa baixada. Resumo final: "Sincronización completa. 1 pedido non puido enviarse, revisa o detalle."
	3a	Erro do servidor durante a baixada (HTTP 5xx).	Mostra "Erro temporal no servidor. Os datos non se actualizaron." e mantén a caché previa intacta.
Rendimiento	Sincronización completa de ata 1.000 clientes + 5.000 produtos + 500 pedidos en menos de 30 segundos con conexión 4G.		
Frecuencia	1-2 veces ao día por comercial (tipicamente ao iniciar a xornada).		
Importancia	Importante.		
Urgencia	Hai presión.		
Comentarios	A sincronización tamén pode dispararse automaticamente (ao iniciar sesión por primeira vez e cada N horas en segundo plano). CU-11 representa só a invocación manual explícita.		

CU-12	Sincronizar Odoo ↔ MySQL		
Descripción	O backend Spring Boot executa periódicamente unha sincronización bidireccional con Odoo: baixa datos mestres modificados e sobe os pedidos e clientes confirmados pendentes.		
Actores	Sistema (primario, scheduler interno), Odoo (secundario, externo).		
Pre condiciones	• O backend Spring Boot está en execución. • Hai conectividade con Odoo (XML-RPC accesible). • A configuración de credenciais de Odoo é válida.		
Post condiciones	• A BD MySQL contén os datos mestres de Odoo actualizados. • Os pedidos confirmados na app foron creados como sale.order en Odoo. • Os clientes novos creados na app foron creados como res.partner en Odoo. • O idOdoo dos rexistros recén sincronizados está actualizado en MySQL		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	O scheduler dispara a tarefa programada (ex: cada hora).	Auténtícase contra Odoo vía XML-RPC (common.authenticate).
	2		Obtén o uid de Odoo e prepara o cliente XML-RPC.
	3		Importa de Odoo os rexistros modificados desde last_sync: clientes (res.partner), produtos (product.product), modos de pagamento, condicións de pagamento (account.payment.term), pedidos do último ano (sale.order + sale.order.line).
	4		Exporta a Odoo: clientes con idOdoo = null créanse como res.partner e recibe o seu id; pedidos con estadoPedido = confirmado e estadoSync = pendente créanse como sale.order e recibe o seu id.
	5		Actualiza a marca interna last_sync ao timestamp actual.
	6		Rexistra no log o resumo (cantidades importadas/exportadas, erros detectados).

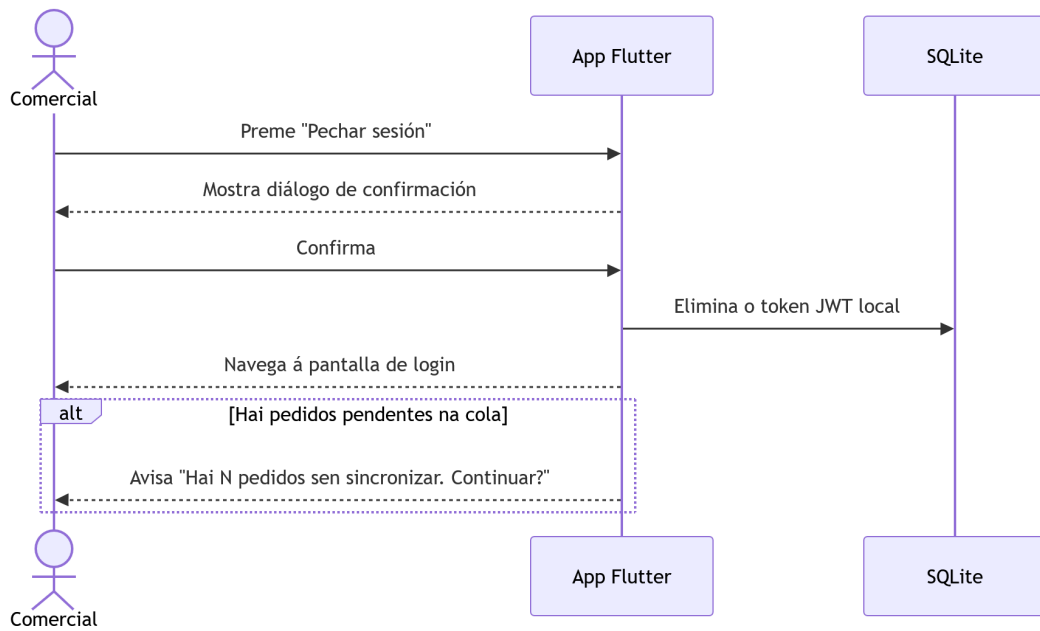
Excepciones	#	Acción (actor)	Reacción (sistema)
	1a	Erro de autenticación con Odoo (credenciais expiradas ou Odoo inaccesible).	Rexistra o erro no log, mantén last_sync sen cambiar e finaliza. A próxima execución reintentará.
	3a	Erro durante importación (Odoo devolve datos malformados).	Rexistra o erro do rexistro afectado, salta ese rexistro e continúa cos seguintes.
	4a	Erro durante exportación dun pedido (Odoo rexeita por validación).	Marca o pedido como estadoSync = erro, almacena o motivo no log e continúa cos seguintes.
	4b	Erro de rede a metade da sincronización.	Rexistra o que se completou e descarta last_sync para reintentar todo na seguinte execución.
Rendimiento	Ciclo completo en menos de 5 minutos para volumes típicos (1.000 clientes, 5.000 produtos, 500 pedidos/ano). Execucións incrementais (delta sync) en menos de 30 segundos.		
Frecuencia	1 execución por hora (configurable). Tamén disparable manualmente desde un endpoint administrativo.		
Importancia	Vital		
Urgencia	Inmediatamente		
Comentarios	Implícase con @Scheduled de Spring. A clase responsable é SyncOdooService; o cliente XML-RPC encapsúlase en OdooClient. Para sincronización incremental compárase fechaModificacion (MySQL) contra write_date (Odoo). A sincronización inicial tras o despregamento debe lanzarse manualmente cunha tarefa específica de importación completa.		

6.4.6 Diagramas de secuencia dos casos de uso

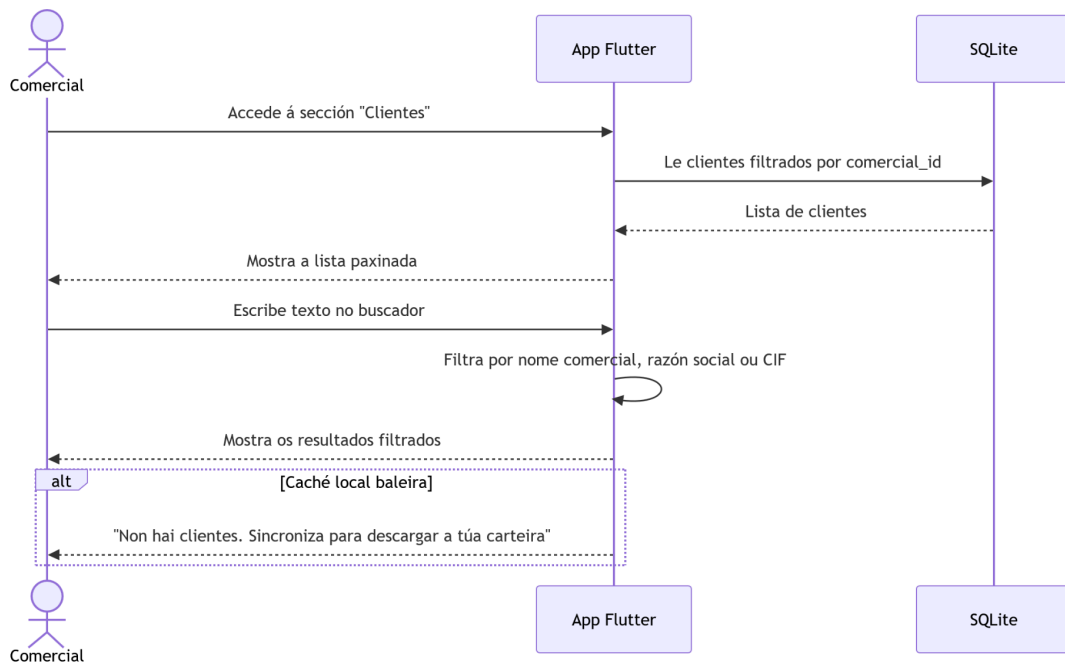
CU-01 — Iniciar sesión



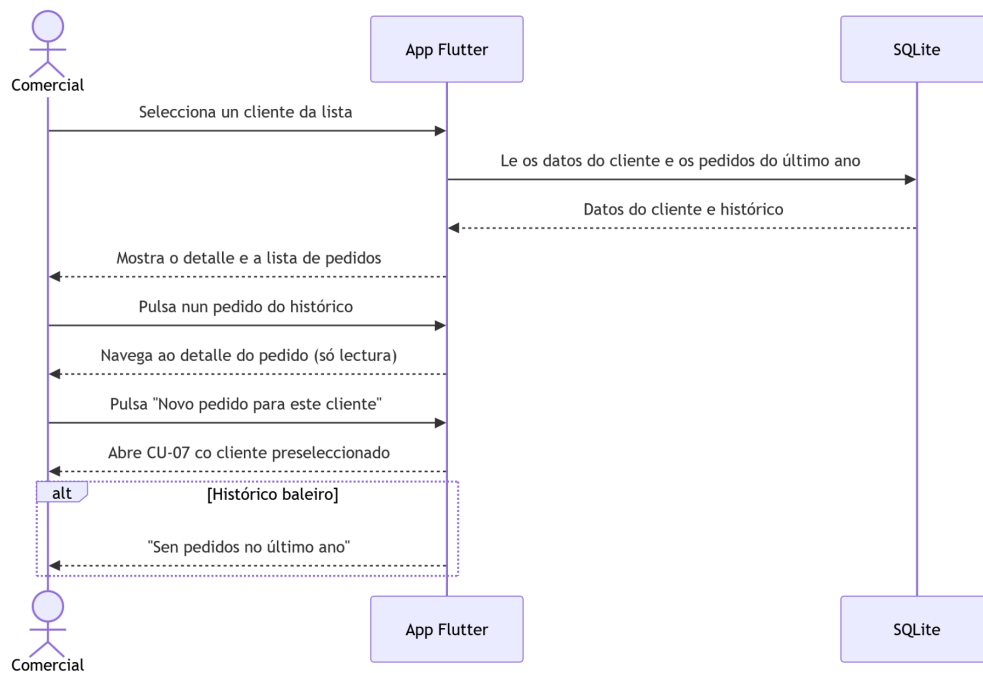
CU-02 — Pechar sesión



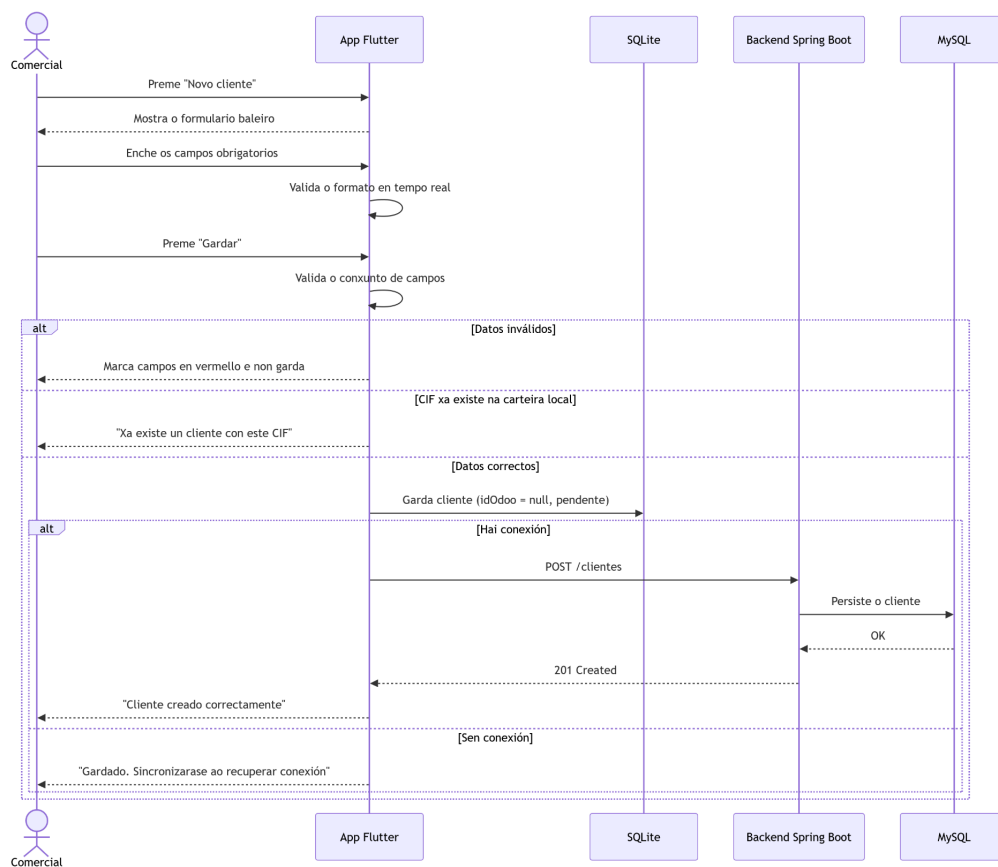
CU-03 — Listar e buscar clientes



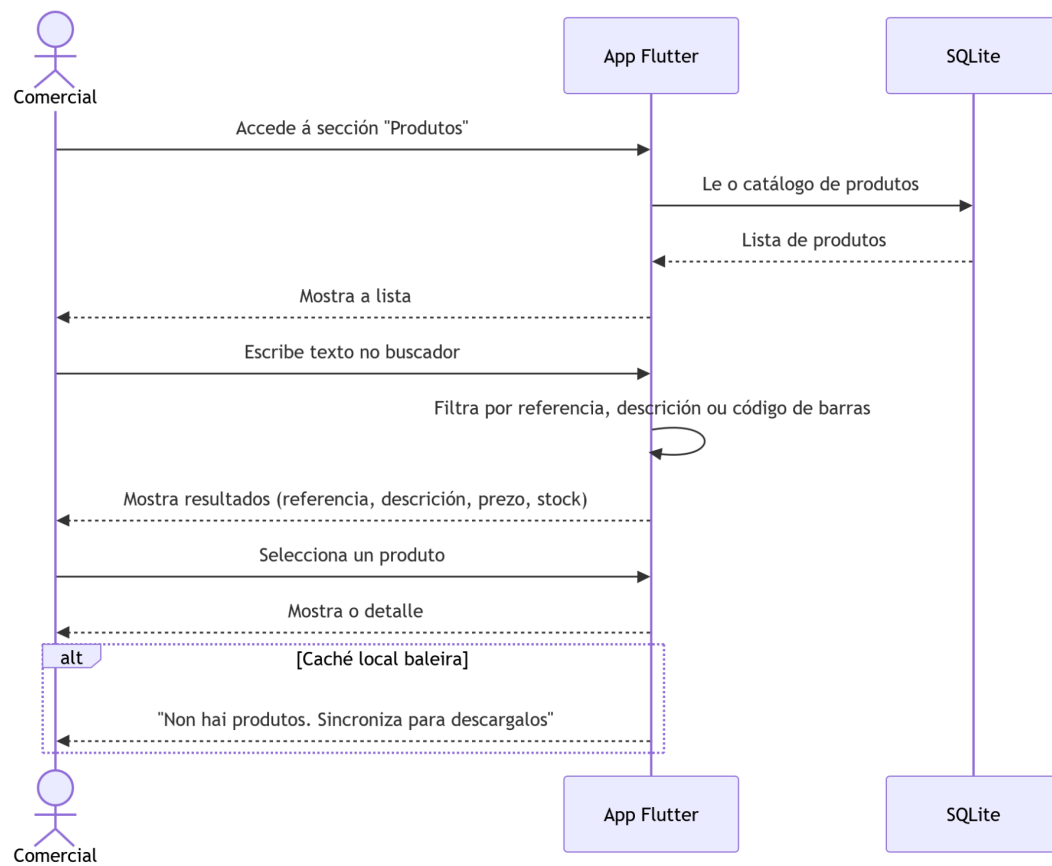
CU-04 — Ver detalle de cliente



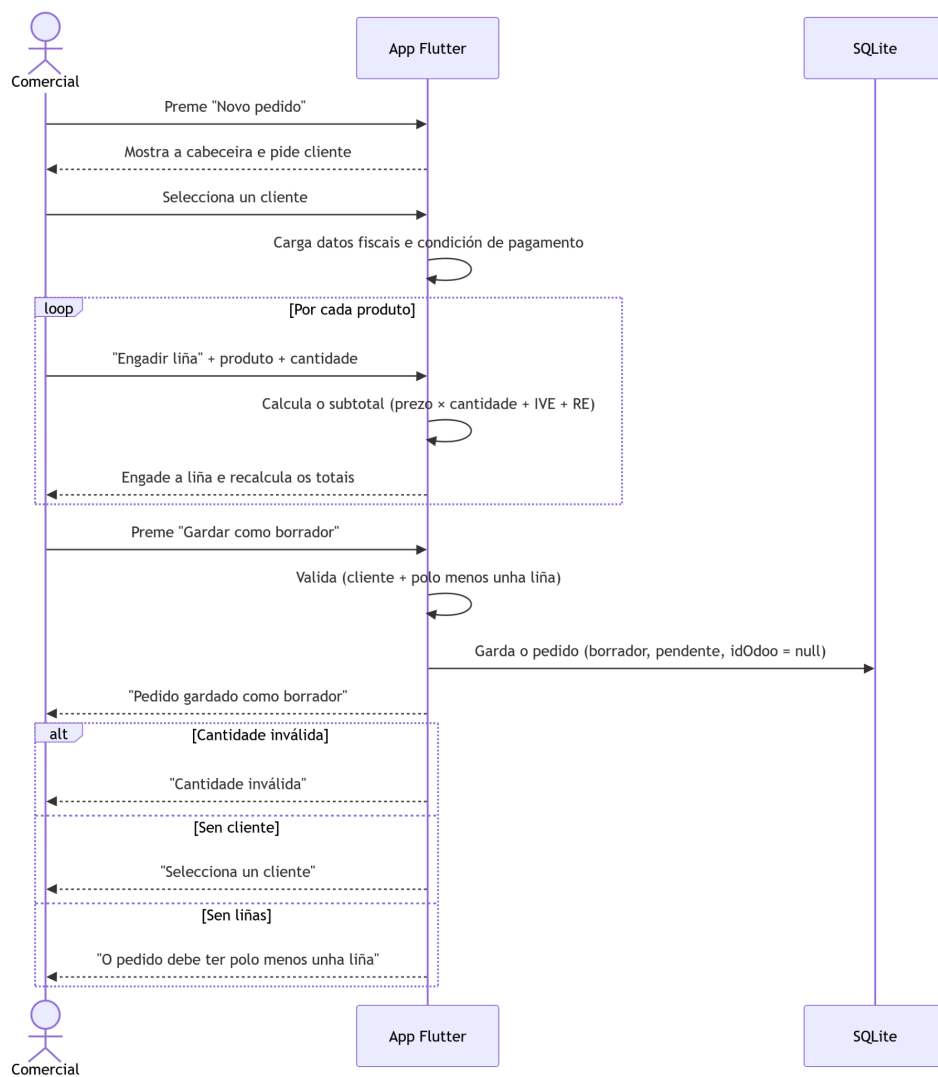
CU-05 — Dar de alta cliente novo



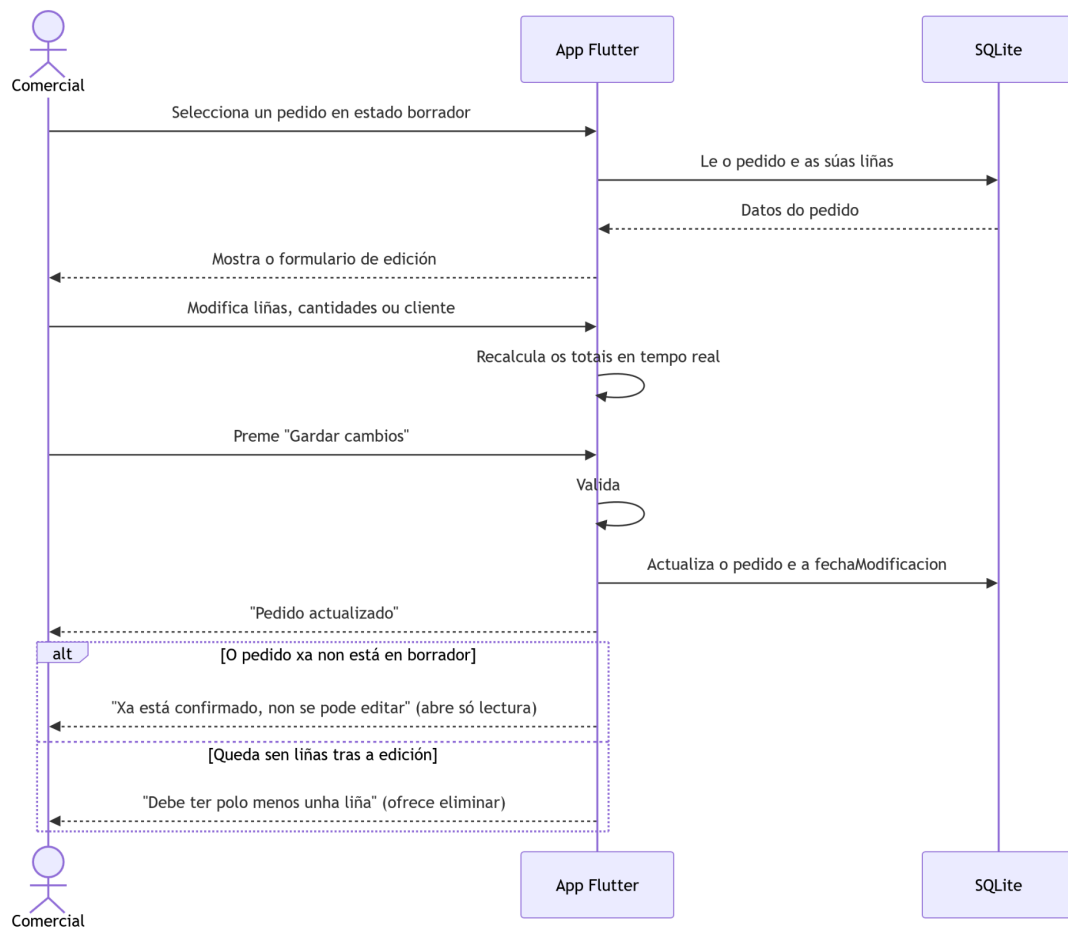
CU-06 — Listar e buscar produtos



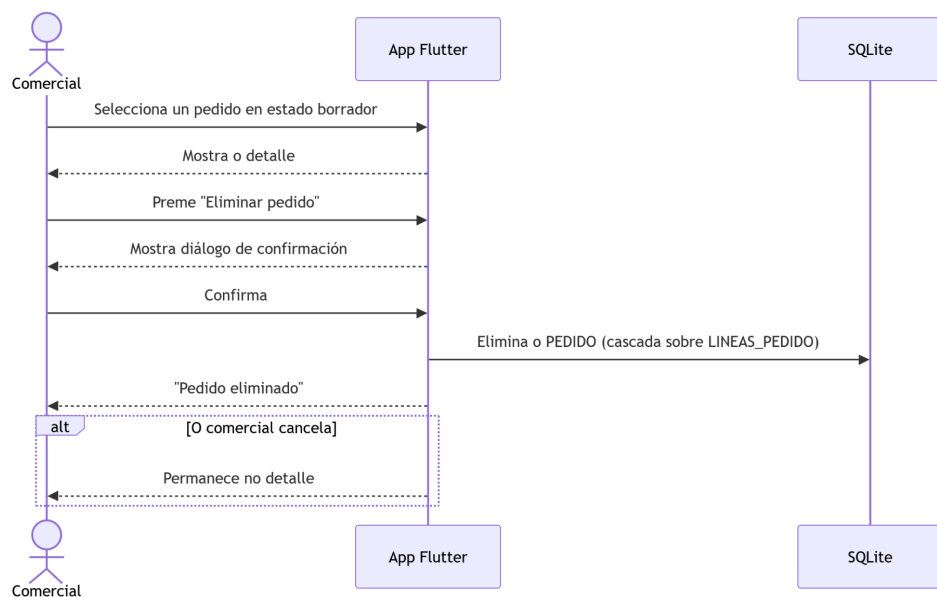
CU-07 — Crear pedido



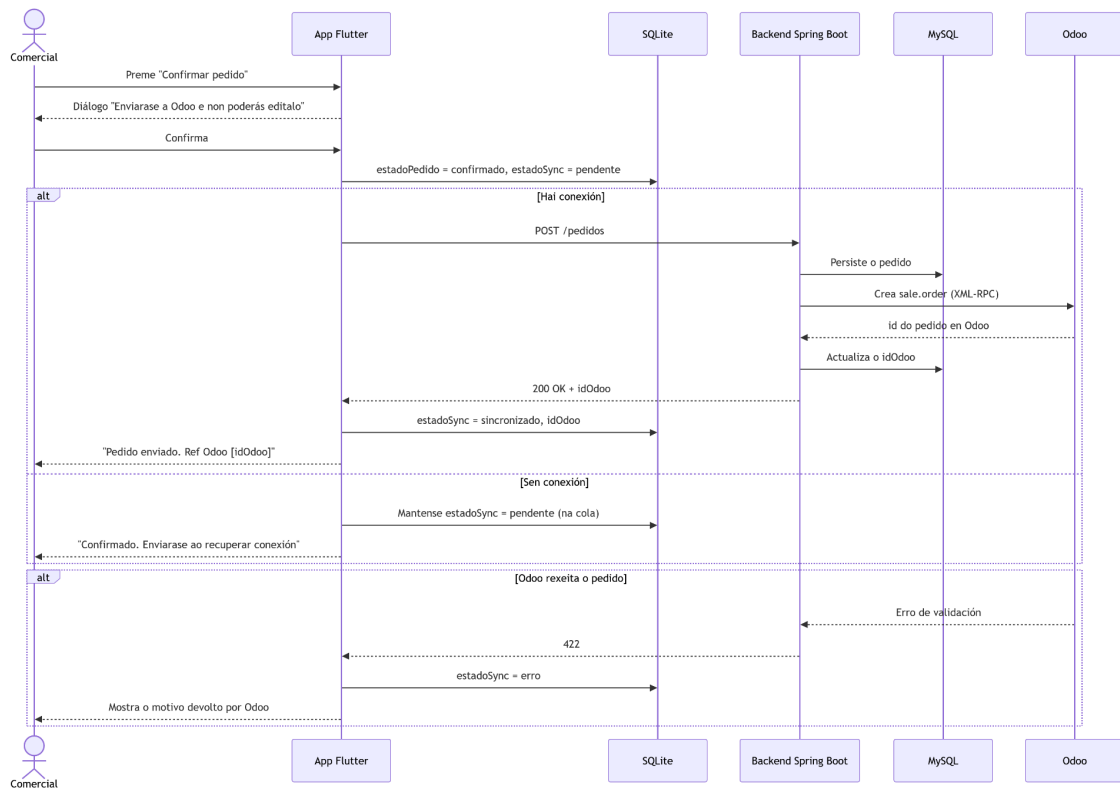
CU-08 — Editar pedido borrador



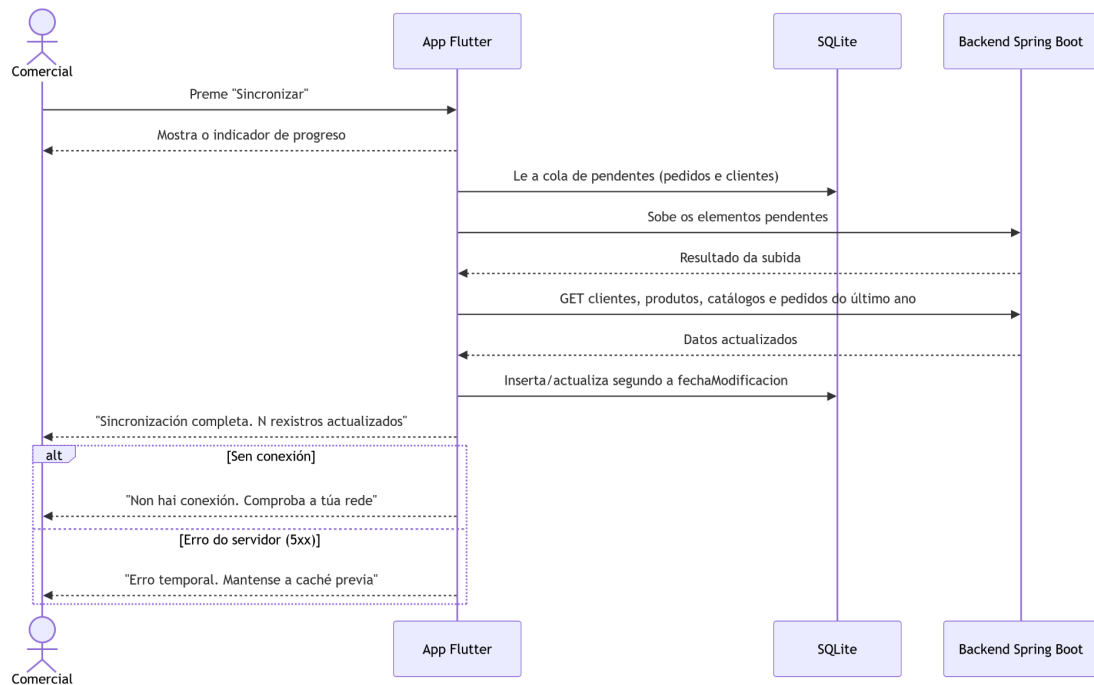
CU-09 — Eliminar pedido borrador



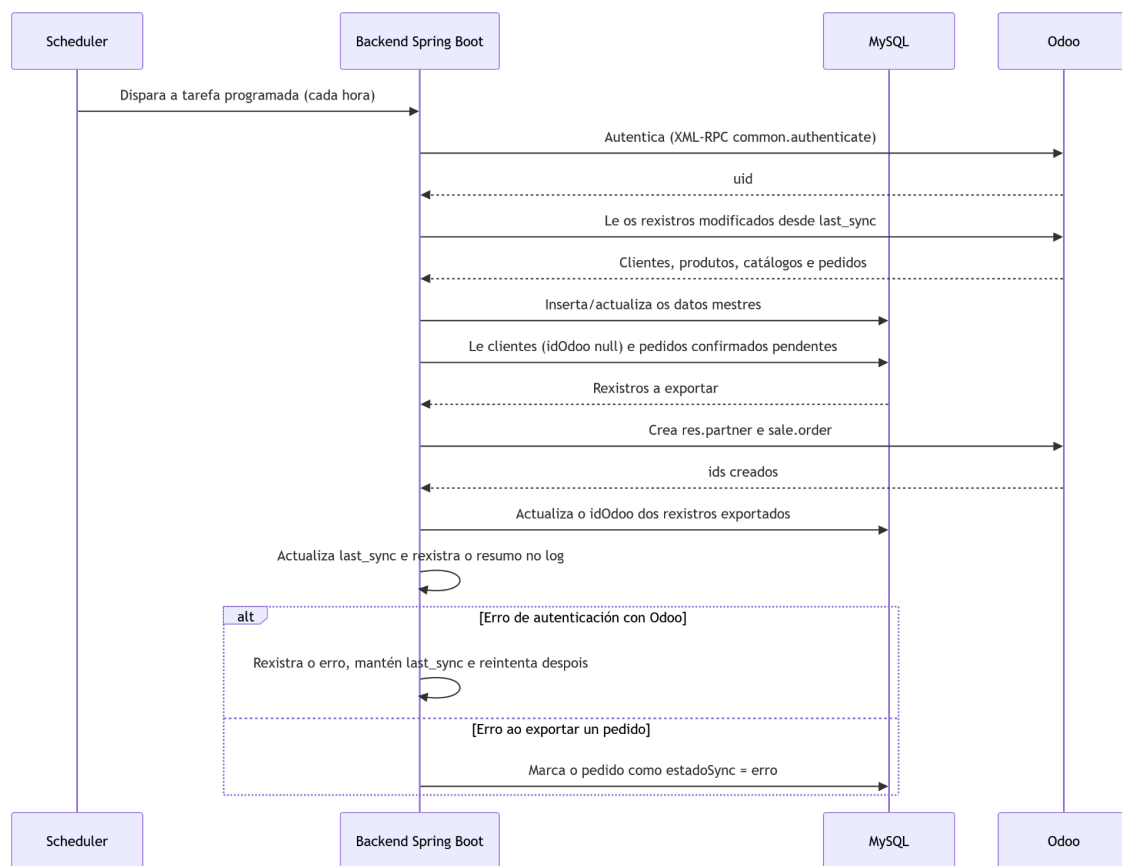
CU-10 — Confirmar e enviar pedido



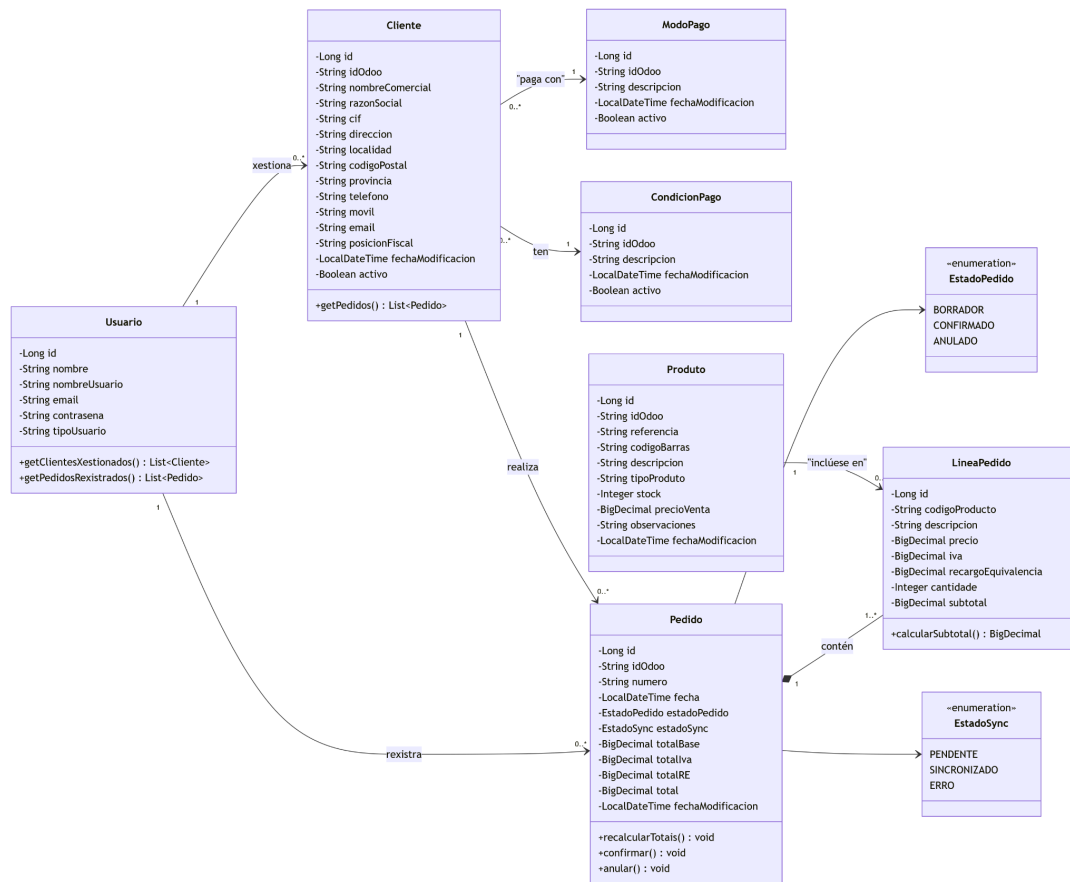
CU-11 — Sincronizar caché desde o servidor



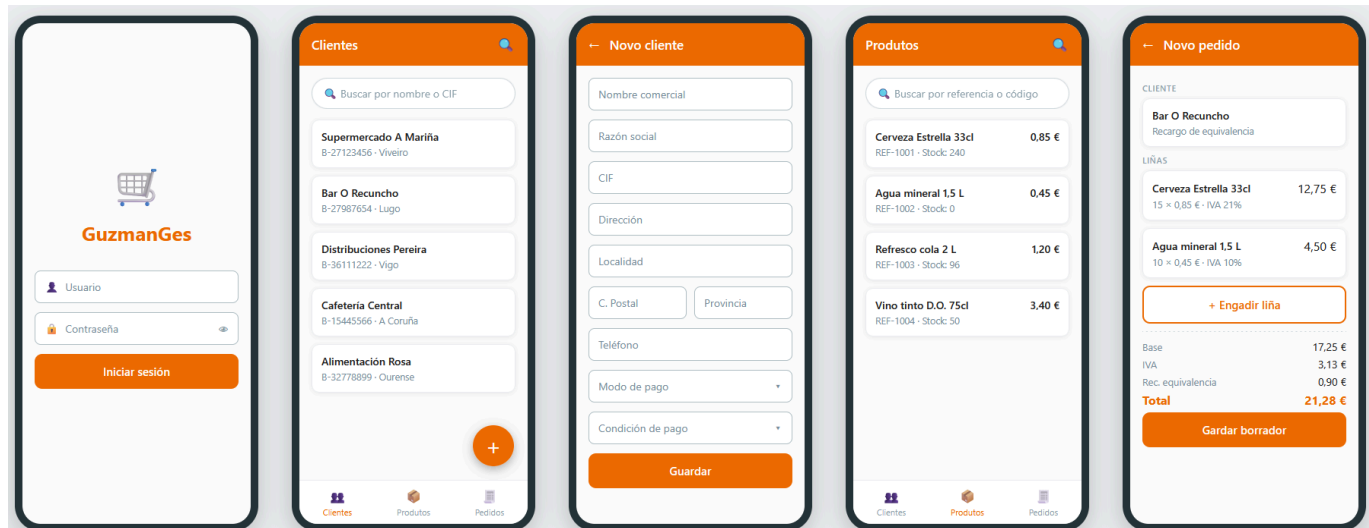
CU-12 — Sincronizar Odoo ↔ MySQL



6.4.7 Diagrama de clases

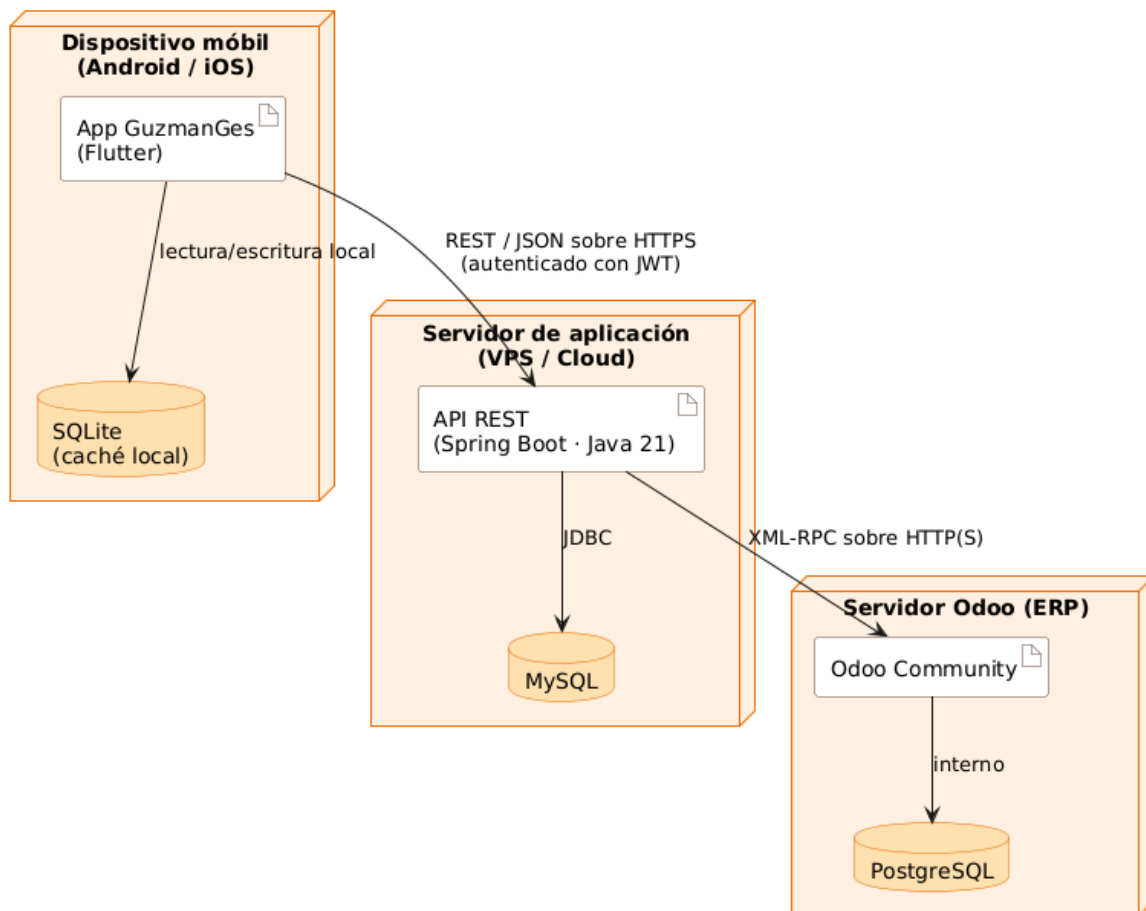


6.4.8 Mockups da interface



7. Desenvolvemento e execución

7.1. Arquitectura e diagrama de despregamento



A solución estrutúrase en **tres pezas independentes** e despregables por separado:

- **Aplicación móbil (Flutter, Dart)**, corrida en dispositivos Android e iOS dos comerciais.
- **Backend REST (Spring Boot, Java 21)**, aloxado nun servidor centralizado da empresa ou no nun cloud.
- **Base de datos MySQL**, que persiste tanto os datos sincronizados desde Odoo como os rexistros novos xerados pola aplicación.

A aplicación móbil non se comunica directamente con Odoo nin coa base de datos: toda a interacción pasa pola API REST, que actúa como capa intermedia. Esta capa é tamén a encargada de coordinar a sincronización con Odoo a través da interface XML-RPC. O diagrama de despregamento (ver sección 6.4) mostra graficamente esta separación de responsabilidades.

A elección desta arquitectura responde a tres motivos principais:

1. **Illamento do ERP:** cambiar de Odoo a outro ERP no futuro non require modificar a aplicación móbil, só a capa de sincronización do backend.
2. **Seguridade:** as credenciais de Odoo nunca residen no dispositivo móbil; só o backend as coñece.

3. **Capacidade offline:** a aplicación móbil traballa contra unha caché SQLite local e sincroniza coa API só baixo demanda, o que permite funcionar en zonas sen cobertura.

7.2. Descrición xeral do funcionamento

O ciclo de vida típico dun pedido na aplicación pasa polas seguintes fases:

1. **Inicio de sesión:** o comercial auténticase contra a API mediante usuario/contrasinal e recibe un token JWT con validez de 24 horas.
2. **Sincronización inicial:** a aplicación descarga os catálogos (modos e condicións de pago), produtos, clientes e pedidos modificados desde a última sincronización. Estes datos quedan persistidos en SQLite no propio dispositivo.
3. **Captura do pedido:** o comercial selecciona o cliente, engade liñas escollendo produtos e cantidades e introduce, opcionalmente, observacións. A aplicación calcula totais provisionais en tempo real considerando o IVA do produto e o recargo de equivalencia segundo a posición fiscal do cliente.
4. **Persistencia local:** o pedido garda en SQLite cun identificador local único (UUID), en estado PENDENTE. Non se require conexión á rede en ningún momento desta fase.
5. **Sincronización ascendente:** cando o comercial pulsa Sincronizar, a aplicación envía os pedidos pendentes á API. Esta gárdalos en MySQL, móstralos en Odoo a través de `sale.order.create` e reescribe os totais cos definitivos que devolve Odoo aplicando a posición fiscal do cliente.
6. **Sincronización descendente:** na mesma operación, a aplicación tamén descarga as novidades desde a última sincronización: pedidos que cambiaron de estado en Odoo, clientes desactivados, etc.

A aplicación filtra os pedidos por usuario autenticado: cada preventa só ve os seus, tanto no dispositivo coma na API. Un usuario ADMIN ten acceso a todos.

7.3. Detalles técnicos resoltos durante o desenvolvemento

Documéntanse aquí os retos máis significativos que apareceron durante o desenvolvemento e a forma en que se resolveron.

7.3.1. Identidade dual para soporte offline

O reto principal do traballo offline é como crear un pedido para un cliente que aínda non existe no servidor central. Se o cliente se da de alta in situ e a continuación se crea un pedido para el, ambos quedan locais sen referencia ao ERP.

A solución implementada é o patrón de **identidade dual**: cada entidade (cliente, pedido) mantén dous identificadores:

- **id_local:** un UUID v4 xerado no dispositivo no momento de creación; está presente sempre.
- **id_servidor:** o identificador numérico que asigna o backend; é nulo ata que o rexistro se sincronizou.

A relación entre pedido e cliente faise polo **id_local** do cliente, non polo identificador do servidor. Cando se executa a sincronización ascendente, o backend primeiro sube o cliente (recibe o seu identificador real) e a continuación sube o pedido, que xa pode resolver correctamente a referencia. Esta cadea de dependencias é robusta fronte a fallos parciais: se a subida do cliente falla, o pedido queda en estado Esperando a que se sincronice o cliente sen producir erros incoherentes.

7.3.2. Reconciliación de borrados na sincronización incremental

A sincronización funciona de forma incremental: a aplicación pide ao servidor o modificado desde a data X. Este esquema funciona ben para altas e modificacións, pero fracasa cos borrados: se un cliente se borra no servidor, simplemente desaparece e a aplicación non recibe ningunha notificación.

A solución foi non realizar nunca borrado físico no backend cando un rexistro desaparece de Odoo. En lugar diso, márcase como **activo=false** no caso dos **clientes** ou como **ANULADO** no caso dos **pedidos**, e actualízase a marca temporal `fechaModificacion`. A aplicación recibe esa actualización na seguinte sincronización incremental e elimina o rexistro do seu propio almacén local. O mesmo principio aplícase aos pedidos cancelados en Odoo (que en Odoo conservan o seu rexistro en estado cancel): a API detéctao no proceso de reconciliación e márcaos en MySQL como anulados.

7.3.3. Validación oficial de CIF/NIF antes do envío

A integración con Odoo rexeita clientes cuxo CIF/NIF non cumpra coa estrutura oficial española. Se a aplicación enviase un cliente con CIF inválido, o erro propagaríase varias capas e devolvería á interface unha mensaxe críptica. Para evitalo, implementouse a validación estrutural completa no propio cliente Flutter, replicando o algoritmo oficial:

- **NIF (persoa física):** 8 díxitos seguidos da letra de control calculada como (número mod 23) sobre a táboa oficial de letras.
- **NIE (estranxeiro):** mesma estrutura cunha letra inicial X, Y ou Z que se substitúe por un dígito equivalente antes do cálculo.
- **CIF (persoa xurídica):** letra inicial dentro do conxunto permitido, sete díxitos, dígito ou letra de control segundo a regra concreta da letra inicial.

A validación execútase no formulario antes do POST /clientes, de modo que o erro se mostra como mensaxe inline xunto ao campo afectado.

7.3.4. Bugs específicos de iOS

Durante a fase de probas en dispositivo físico iOS detectáronse tres comportamentos que non se reproducían no simulador nin en Android:

- O **teclado numérico de iOS** non incorpora un botón Hecho, e o usuario podía quedar bloqueado se o campo de entrada acababa fóra da zona visible. Solucionouse engadindo un **GestureDetector** que pecha o teclado ao tocar fóra do campo e activando o modo **ScrollViewKeyboardDismissBehavior.onDrag** na lista.
- Ao navegar dun formulario a un selector modal e volver, iOS restauraba o foco ao último TextField editado e reabría o teclado. Resolveuse forzando `FocusManager.instance.primaryFocus?.unfocus()` e `SystemChannels.textInput.invokeMethod('TextInput.hide')` tanto antes como despois da navegación.
- A xeración do IPA sen sinatura non é trivial: **flutter build ios --no-codesign** produce só o .app, non o .ipa. Foi necesario empaquetar manualmente o .app dentro dunha carpeta Payload/ e comprimilo a .ipa para que ferramentas como Sideloadly puidesen re-asinalo coa Apple ID gratuíta do usuario final.

7.3.5. Dockerización e despregamento reproducible

Para evitar que cada despregamento dependa de configuracións concretas da máquina anfitrión, todo o backend (API + MySQL) empaquetouse en contenedores Docker. O Dockerfile da API é multi-stage:

- **Etapas de build:** imaxe eclipse-temurin:21-jdk-alpine que descarga dependencias Maven nunha capa cacheable e compila o JAR.
- **Etapas de runtime:** imaxe eclipse-temurin:21-jre-alpine, sen ferramentas de compilación, executando como usuario non-root por motivos de seguridade.

O **docker-compose.yml** orquestra os dous servizos. MySQL non se expón ao host por defecto (só interno), aínda que se proporciona un ficheiro **docker-compose.override.example.yml** para o caso de necesitar acceso desde un cliente gráfico durante o desenvolvemento. A API dispón dun healthcheck contra `/actuator/health` que permite a Docker e a Portainer detectar caídas e reiniciar o contedor automaticamente.

7.3.6. Documentación da API con OpenAPI/Swagger

A integración co frontend e a comunicación con desenvolvedores externos exige unha documentación actualizada e accesible. Decidiuse non mantela a man, senón xeralá automaticamente desde o código mediante Springdoc OpenAPI 3, que inspecciona as anotacións Spring (`@RestController`, `@RequestMapping`, `@RequestBody`, etc.) e produce unha especificación OpenAPI 3 acompañada da interface Swagger UI.

Configurouse adicionalmente un esquema de seguridade bearerAuth que permite pegar un JWT obtido de `/auth/login` e probar calquera endpoint protexido directamente desde o navegador. As rutas `/swagger-ui.html` e `/v3/api-docs` están abertas sen autenticación para que a documentación sexa accesible sen necesidade de iniciar sesión.

7.3.7. Integración continua e xeración automática de instaladores

Estableceuse un workflow de GitHub Actions (`.github/workflows/release.yml`) que xera automaticamente os tres artefactos de instalación cada vez que se publica un tag `vX.Y.Z` no repositorio:

1. **Imaxe Docker da API**, exportada con `docker save | gzip` como `guzmanges-api-vX.Y.Z.tar.gz`.
2. **APK Android**, compilado con `flutter build apk --release`.
3. **IPA iOS sen sinatura**, xerado nun runner macOS con `flutter build ios --no-codesign` e empaquetado a man.

Os tres ficheiros adxúntanse a unha release que se crea inicialmente como borrador para permitir verificación previa antes da publicación. O workflow tamén pode lanzarse manualmente desde a interface de GitHub Actions cunha versión introducida pola persoa usuaria, o que facilita probas de release sen necesidade de empurrar tags.

8. Conclusións e reflexións

8.1. Valoración global do proxecto

GuzmanGes acadou os obxectivos definidos ao inicio do proxecto: dispón dunha aplicación móbil multiplataforma que permite ao preventa consultar clientes, produtos e pedidos, dar de alta clientes novos

e crear pedidos completos sen depender da conexión, e que se integra de forma bidireccional cun ERP Odoo Community.

A arquitectura adoptada (frontend Flutter, backend Spring Boot, base de datos MySQL, integración XML-RPC con Odoo) demostrou ser apropiada para o caso de uso: separa responsabilidades de forma clara, permite substituír cada compoñente por separado e proporciona a flexibilidade necesaria para futuras extensións. A capacidade de traballo sen conexión, implementada mediante caché SQLite local e cola de pendentes, funciona correctamente nos escenarios de proba realizados.

A despregamento mediante Docker reduce a complexidade da posta en marcha: o instalador típico que se atoparía nun cliente real (unha empresa cun servidor propio na sede ou un VPS) precisa só Docker e o ficheiro `docker-compose.yml` para deixar o sistema operativo en menos dunha hora. Os instaladores móbiles (APK e IPA) xeranse automaticamente con cada release, eliminando o traballo manual de empaketado.

8.2. Aprendizaxes

O proxecto supuxo enfrontarse a varios desafíos novos que enriqueceron considerablemente a formación recibida no ciclo:

- **Deseño offline-first:** pensar primeiro no comportamento sen conexión, e logo encaixar a sincronización por riba, é fundamentalmente distinto a deseñar unha aplicación online que se adapte a estar sen rede. O patrón de identidade dual e a resolución de dependencias en cola foron descubrimentos prácticos durante o desenvolvemento, non decisións iniciais.
- **Integración con sistemas externos:** traballar contra Odoo a través de XML-RPC esixiu lecturas profundas da súa documentación, comprensión do seu modelo de datos (posicións fiscais, recargo de equivalencia, ciclo de vida de `sale.order`) e adaptación da capa de sincronización a particularidades como o formato exacto de datas ou a representación HTML do campo `note`.
- **Particularidades de iOS:** aínda que Flutter abstrae a maior parte das diferenzas entre plataformas, hai detalles específicos de cada SO que só aparecen ao probar en dispositivo real. Esta experiencia reforzou a importancia de incluír probas en hardware desde fases temperás.
- **DevOps e empaketado:** a CI/CD con GitHub Actions abordouse ao final do proxecto, unha vez estabilizado o alcance funcional. Aínda que o resultado (xeración automática de APK, IPA e imaxe Docker en cada release) cumpre o seu propósito, a experiencia demostrou que introducila antes tería simplificado a verificación continua de cada cambio e evitado fricción no peche. É unha lección clara para futuros proxectos: tratar o pipeline de empaketado como infraestrutura inicial, non como tarefa final.
- **Documentación como parte do desenvolvemento:** redactar os manuais e a propia memoria de forma progresiva, paralela ao código, aforra un esforzo considerable fronte á alternativa de facelo todo ao final.

8.3. Dificultades atopadas

Entre as dificultades técnicas máis salientables:

- **Compatibilidade de versións recentes:** o uso de Spring Boot 4.0 e Jackson 3 trouxo cambios de namespaces respecto a versións anteriores (`com.fasterxml.jackson` → `tools.jackson`) que obrigaron a corrixir imports en distintos puntos. A documentación destas versións aínda non é tan extensa como a das anteriores.
- **Carreiras de condición na sincronización:** ao implementar o borrado de datos locais detectouse unha carreira entre o cerre da base de datos SQLite e unha lectura asíncrona disparada polo callback do logout. Resolveuse reordenando as operacións para que o pechamento da base de datos ocorra antes de calquera nova operación.

- **Asinado e distribución iOS:** a obtención dun IPA instalable sen conta Apple Developer de pagamento limitou a forma de distribución a sideloading con re-firma cada 7 días. É unha limitación coñecida que se documentou claramente.
- **Sincronización Odoo bidireccional con posición fiscal:** o cálculo do recargo de equivalencia depende dunha posición fiscal específica configurada en Odoo, e a forma de comunicalo entre aplicación, backend e ERP requireu varias iteracións ata atopar un esquema robusto (totais provisionais na app → totais definitivos despois da confirmación de Odoo).

8.4. Limitacións actuais

O proxecto cumpre o alcance definido, pero presenta as seguintes limitacións coñecidas:

- A **xestión de usuarios da aplicación** non está dispoñible desde a propia aplicación móbil: hai que recorrer aos endpoints REST protexidos para ADMIN. O motivo é que se quixo manter a app centrada no fluxo de preventa.
- A **ficha de produto** non inclúe foto nin vista en formato catálogo visual. Os produtos son meramente filas con descrición, prezo e stock.
- Os **pedidos creados** desde a app **non admiten descontos** por liña nin globais: o prezo enviado a Odoo é sempre o de tarifa. Tampouco se poden consultar nin aplicar as ofertas e promocións que Odoo ten configuradas (pricelists, descontos por volume, campañas), polo que calquera condición especial debe pactarse fóra da app.
- A **sincronización** só se executa **baixo demanda do usuario**. Non hai un fluxo de sincronización en segundo plano cando se detecta Wi-Fi.
- O **IPA distribúese sen sinatura oficial**, polo que require sideloading cun límite de 7 días por re-firma.
- Non existe un **fluxo público de recuperación de contrasinal** por email: a baixa ou cambio de contrasinal debeo facer un ADMIN no backend.

8.5. Liñas futuras

Implementar un modelo de licenzas por usuario que permita distribución comercial da aplicación.

- Implementar un modelo de licenzas por usuario que permita distribución comercial da aplicación.
- Engadir a distribución oficial vía Play Store e App Store con conta Apple Developer real.
- Crear un endpoint público de recuperación de contrasinal con verificación por email.
- Sincronización en segundo plano cando o dispositivo detecte Wi-Fi.
- Soporte multi-empresa nun mesmo despregue do backend.
- Inclusión de fotos nos produtos e vista de catálogo visual.
- Liñas de pedido con tipos especiais (sección, nota) cando Odoo o expoña.
- Soporte de descontos nos pedidos (por liña e globais) e integración coas pricelists e promocións de Odoo, de xeito que o comercial poida aplicar as ofertas xa configuradas no ERP sen saír da app.

9. Bibliografía e Webgrafía

Referencias en formato APA. Inclúense as fontes oficiais consultadas durante o desenvolvemento, agrupadas por tecnoloxía.

9.1. Frontend (Flutter, Dart)

Flutter. (2025). *Flutter documentation*. Recuperado de <https://docs.flutter.dev/>

Dart. (2025). *Dart language tour*. Recuperado de <https://dart.dev/language>

Flutter Team. (2025). *State management with Provider*. Recuperado de <https://docs.flutter.dev/data-and-backend/state-mgmt/simple>

Tekartik. (2025). sqflite — *SQLite plugin for Flutter*. Recuperado de <https://pub.dev/packages/sqflite>

Wendux. (2025). Dio — *A powerful HTTP client for Dart*. Recuperado de <https://pub.dev/packages/dio>

9.2. Backend (Spring Boot, Java)

Pivotal Software. (2025). *Spring Boot 4.0 reference documentation*. Recuperado de <https://docs.spring.io/spring-boot/docs/current/reference/html/>

Pivotal Software. (2025). *Spring Security reference*. Recuperado de <https://docs.spring.io/spring-security/reference/>

Pivotal Software. (2025). *Spring Data JPA — Reference documentation*. Recuperado de <https://docs.spring.io/spring-data/jpa/reference/>

JWT. (2025). Java JWT — *JSON Web Token for Java*. Recuperado de <https://github.com/jwt/jwt>

FasterXML. (2025). *Jackson 3 release notes*. Recuperado de <https://github.com/FasterXML/jackson/wiki/Jackson-Release-3.0>

9.3. Base de datos

Oracle. (2025). *MySQL 8.0 reference manual*. Recuperado de <https://dev.mysql.com/doc/refman/8.0/en/>

SQLite Consortium. (2025). *SQLite documentation*. Recuperado de <https://sqlite.org/docs.html>

9.4. Integración con Odoo

Odoo S.A. (2025). *Odoo 18 documentation — Web services API*. Recuperado de https://www.odoo.com/documentation/18.0/developer/reference/external_api.html

Odoo Community Association. (2025). *OCA repositories*. Recuperado de <https://github.com/OCA>

9.5. DevOps e CI/CD

Docker Inc. (2025). *Docker documentation*. Recuperado de <https://docs.docker.com/>

Docker Inc. (2025). *Docker Compose v2 specification*. Recuperado de <https://docs.docker.com/compose/>

GitHub. (2025). *GitHub Actions documentation*. Recuperado de <https://docs.github.com/actions>

Springdoc. (2025). *Springdoc OpenAPI v2 documentation*. Recuperado de <https://springdoc.org/>

OpenAPI Initiative. (2025). *OpenAPI Specification 3.1.0*. Recuperado de <https://spec.openapis.org/oas/v3.1.0>

9.6. Distribución iOS

Sideloadly. (2025). Sideloadly — *Sideload iOS apps*. Recuperado de <https://sideloadly.io/>

Apple Inc. (2025). *Apple Developer Documentation*. Recuperado de <https://developer.apple.com/documentation/>

10. Anexo I – Manual técnico de instalación ou posta en marcha

11. Anexo II – Documentación de uso (manuais usuario)

12. Anexo III – Outra documentación

Calquera material adicional relevante (documentación da aplicación, manual de uso, capturas de pantalla, fragmentos de código, etc.).